



PAPER • OPEN ACCESS

QCPINN: quantum-classical physics-informed neural networks for solving PDEs

To cite this article: Afrah Farea *et al* 2025 *Mach. Learn.: Sci. Technol.* **6** 045053

View the [article online](#) for updates and enhancements.

You may also like

- [Condition monitoring of levitation systems for maglev trains using Bayesian physics-informed neural networks](#)
Lin Fan, Guobin Lin, Qinghua Zheng et al.
- [Physics-informed neural networks for modeling astrophysical shocks](#)
S P Moschou, E Hicks, R Y Parekh et al.
- [Physics-informed neural networks for physiological signal processing and modeling: a narrative review](#)
Anni Zhao, Davood Fattahi and Xiao Hu



PAPER

OPEN ACCESS

RECEIVED
10 April 2025REVISED
21 October 2025ACCEPTED FOR PUBLICATION
6 November 2025PUBLISHED
25 November 2025

Original Content from
this work may be used
under the terms of the
Creative Commons
Attribution 4.0 licence.

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



QCPINN: quantum-classical physics-informed neural networks for solving PDEs

Afrah Farea^{1,*} , Saiful Khan² and Mustafa Serdar Celebi¹ ¹ Computational Science and Engineering Department, Istanbul Technical University, Istanbul 34469, Türkiye² Scientific Computing, Rutherford Appleton Laboratory, STFC, Harwell OX11 0QX, United Kingdom

* Author to whom any correspondence should be addressed.

E-mail: farea16@itu.edu.tr, saiful.khan@stfc.ac.uk and mscelebi@itu.edu.tr**Keywords:** quantum computing, quantum machine learning, neural networks, physics-informed neural network, PINN, partial differential equation, and PDE

Abstract

Physics-informed neural networks (PINNs) have emerged as promising methods for solving partial differential equations (PDEs) by embedding physical laws within neural architectures. However, these classical approaches often require a large number of parameters to achieve reasonable accuracy, particularly for complex PDEs. In this paper, we present a quantum-classical PINNs (QCPINNs) that combines quantum and classical components, allowing us to solve PDEs with significantly fewer parameters while maintaining comparable accuracy and convergence to classical PINNs. We systematically evaluated two quantum circuit architectures across various configurations on five benchmark PDEs to identify optimal QCPINN designs. Our results demonstrate that the QCPINN achieves stable convergence and comparable accuracy, while using only 10%–30% of the trainable parameters required by classical PINNs. This approach also results in a significant reduction in the relative L_2 error for Helmholtz, Klein–Gordon, and Convection–diffusion equations, with reduction ranging from 4% to 64% across various fields. These findings demonstrate the potential of parameter efficiency and solution accuracy in physics-informed machine learning, allowing for a substantial decrease in model complexity without compromising solution quality.

1. Introduction

Recent advancements in machine learning have transformed approaches to solving partial differential equations (PDEs), with physics-informed neural networks (PINNs) emerging as an innovative technique. PINNs embed physical laws directly into neural network (NN) architectures, eliminating the need for labeled training data while strictly enforcing physical constraints. This approach has demonstrated considerable success in modeling various physical phenomena across scientific and engineering domains [1–4, 5]. Although they hold great potential, research is underway to improve the accuracy of PINNs in modeling different PDEs and physical systems. They often encounter challenges with stiff systems, capturing steep gradients, and efficiently navigating high-dimensional parameter spaces for reasons such as memory requirements, slow convergence, or when calculating high-order or high-dimensional derivatives [6, 7]. While increasing model expressivity through additional parameters can alleviate these challenges, this approach introduces new problems: increased computational cost, risk of overfitting, and unstable convergence behavior during training [8, 9]. Such a trade-off between model complexity, generalization, and convergence ability is a fundamental challenge in NN design. Quantum computing presents a promising paradigm for overcoming these limitations. With its ability to process high-dimensional data and exploit quantum principles such as superposition and entanglement, quantum algorithms offer unique advantages for solving complex computational problems. For instance, the Harrow–Hassidim–Lloyd (HHL) algorithm has demonstrated theoretical exponential speedups for

specific linear systems, while variational quantum algorithms (VQA) provide practical near-term solutions [10–12]. Moreover, recent developments [13–16] indicate that quantum NNs (QNNs) possess an expressive power that could significantly enhance the learning capabilities for modeling complex physical systems.

Although both PINNs and quantum computing have advanced independently, their integration remains largely unexplored. Several preliminary efforts have emerged: Sedykh *et al* proposed HQPINN [17], focusing on computational fluid dynamics in 3D Y-shaped mixers; Dehaghani *et al* introduced QPINN [18], integrating a dynamic quantum circuit with classical computing methodologies for optimal control problems; Trahan *et al* [19] investigated both purely quantum and hybrid PINNs. Leong *et al* proposed HQPINN [20], which is inspired by Fourier neural operators (FNOs). However, existing research has not comprehensively established whether quantum-enhanced models can improve upon purely classical PINNs across a broad range of PDEs efficiently and reliably.

We investigate the limitations of traditional PINNs by integrating quantum computing circuits with classical NNs. We propose that such a hybrid approach can leverage complementary strengths: classical computing's robustness and quantum computing's enhanced expressivity with inherent parallelism. We believe this method will allow us to solve a wide range of PDEs with similar or better accuracy while requiring significantly fewer trainable parameters than classical models.

To this end, we introduce a hybrid architecture named quantum-classical PINNs (QCPINNs). To search for a suitable QCPINN architecture and its configurations, we systematically evaluate multiple dimensions of the design space: two quantum circuit architectures: (a) discrete-variable (DV) qubit-based circuits with configurations, such as four circuit topologies: Alternate, Cascade, Cross-mesh, and Layered; and two embedding schemes: Amplitude and Angle; (b) continuous-variable (CV) circuits with configurations, including two measurement schemes: number and quadrature; three nonlinear operations: Kerr, cubic phase, and cross-Kerr; and two parameterizations: phase-free (magnitude) and full (magnitude and phase). We evaluate the performance of these QCPINN models across various PDEs, such as the Helmholtz and lid-driven cavity equations, while also extending our analysis to the Klein–Gordon, Wave, and Convection–diffusion equations. Our comprehensive assessment compares (a) solution accuracy, (b) convergence rates, (c) parameter efficiency (d) time, and (f) memory between classical PINNs and QCPINN variants.

To ensure reproducibility and facilitate collaborative advancement, we have made all components of QCPINN open-source code and publicly available on GitHub at <https://github.com/afrah/QCPINN> [21].

2. Related work

In the domain of PDE solvers, quantum algorithms can be broadly categorized into two primary approaches: (a) exclusively quantum algorithms and (b) hybrid quantum-classical methods.

2.1. Exclusively quantum algorithms

Exclusively quantum algorithms for solving PDEs typically leverage unitary operations and quantum state embedding techniques to achieve a computational advantage. In contrast to NNs, exclusively quantum algorithms use fixed quantum circuit structures with no trainable parameters. The celebrated HHL algorithm is an example of this category, which provides exponential speedups to solve linear systems under specific conditions [22]. Extensions of the HHL algorithm have been applied to solve sparse linear systems derived from discretized linear [23, 24] or nonlinear [25] PDEs, leveraging numerical techniques such as the linear multistep method [26], Chebyshev pseudospectral method [27], and Taylor series approximations [28]. In fluid dynamics applications, lattice-gas quantum models have been examined for directly solving PDEs [29, 30] or PDEs derived from ODEs through methods such as the quantum amplitude estimation algorithm [31, 32].

Despite these advancements, purely quantum algorithms for solving PDEs encounter practical challenges, such as requirements for quantum error correction [33–35], limitations in circuit depth [30, 36, 37], and bottlenecks in classical data embedding [38–40]. These obstacles, among others, limit the near-term viability of exclusively quantum PDE solvers on today's noisy intermediate-scale quantum (NISQ) devices.

2.2. Hybrid quantum-classical approaches

Hybrid quantum-classical approaches have emerged as promising solutions to overcome the limitations of exclusively quantum algorithms. These methods combine the nonlinearity, maturity, and stability of

classical computation with the enhanced capacity, expressivity, and inherent parallelism of quantum systems. While classical components can efficiently handle pre-/post-processing, optimization, and error stabilization, quantum layers can capture intricate patterns, solve sub-problems, and accelerate specialized computations.

Within this hybrid paradigm, QNNs, also referred to as parameterized quantum circuits (PQC), have demonstrated universal approximation capabilities for continuous functions when implemented with sufficient depth, width, and well-designed quantum feature maps. For instance, Pérez-Salinas *et al* [41] showed that single-qubit networks with iterative input reuploading can approximate bounded complex functions, while Goto *et al* [42] established universal approximation theorems for quantum feedforward networks. Schuld *et al* [14] revealed that the outputs of PQC (with suitable data embedding and circuit depth) can be expressed as truncated Fourier series, thereby providing theoretical support for the universal approximation ability of QNNs. Liu *et al* [43] provided formal evidence of quantum advantage in function approximation. Killoran *et al* [44] demonstrate the universality of the CV quantum models by showing that many classical NNs can be effectively emulated on a photonic quantum computer. These developments not only support applications in solving PDEs and optimization tasks but also extend to other computational tasks, such as quantum chemistry [45], material sciences [46, 47], and pattern recognition [48], to name a few.

A well-known example of this integrated approach is the VQA or variational quantum circuits (VQC). These algorithms employ PQC with tunable gates optimized through classical feedback loops. VQA implementations for solving PDEs include the variational quantum eigensolver [49, 50], variational quantum linear solver [51], and the quantum approximate optimization algorithm [52], as well as emerging methods like Differentiable Quantum Circuits [53, 54], quantum kernel methods [55], and quantum nonlinear processing algorithms [56]. The aforementioned methods primarily utilize qubit-based quantum computing, which represents the predominant paradigm in quantum computing literature. However, CV quantum computing [57] offers an alternative computational model that leverages continuous degrees of freedom, such as quadratures and electromagnetic field momentum. For consistency, we refer to qubit-based models as DV circuit models throughout this work.

Recently, VQA has been extended to Physics-informed QNNs [17–20, 58], broadening their applicability across physics and engineering domains. These hybrid methods offer several distinct advantages when implemented within the PINN framework.

Firstly, these models enable the flexible configuration of inputs and outputs independent of qubit/qumode counts or quantum layer depth, making them particularly effective for scenarios where the input dimension (e.g., temporal and spatial coordinates) differs from output dimensions (e.g., dependent variables like velocity, pressure, and force). Secondly, hybrid approaches facilitate increased trainable parameter counts without the need for deeper quantum circuits, additional qubits, or higher cutoff dimensions in CV quantum systems. These factors are particularly crucial in the NISQ era, where hardware constraints impose strict limits on circuit depth and coherence times, making deeper quantum networks computationally expensive and impractical [59, 60]. Thirdly, these qualities are especially advantageous for PINNs, where shallow network architectures are favored to optimize the balance between computational efficiency and solution accuracy [9, 61–63]. Finally, a substantial benefit of such hybrid models is the integration of automatic differentiation within both classical and quantum networks. This capability, supported by contemporary software packages like PennyLane [64] and TensorFlow Quantum [65], enables the automatic computation of derivatives as variables are directly incorporated into the computational graph. This approach simplifies PINN implementation and facilitates the efficient handling of complex physics-informed loss functions and higher-order derivatives.

Our work is related to the approaches in [17–20], but with several key differences. While HQPINN [17] focuses specifically on computational fluid dynamics in 3D Y-shaped mixers using a fixed quantum depth with an infused layer structure, the QPINN [18] integrates a dynamic quantum circuit combining Gaussian and non-Gaussian gates with classical computing methodologies to solve optimal control problems. In addition, the model proposed in [19] investigates both purely quantum and hybrid PINNs, highlighting parameter efficiency but relying on limited PQC networks and addressing primarily simple PDEs. Recently proposed HQPINN [20], inspired by the FNO [66], introduces a parallel hybrid architecture to separately model harmonic and non-harmonic features before combining them for the output. The present work addresses general PDE solving with multiple quantum circuit configurations and explicit emphasis on parameter efficiency.

3. Background

3.1. PINNs

In the present work, we consider a general PDE of the form:

$$\begin{aligned}\mathcal{D}[u(\mathbf{x}); \alpha] &= f(\mathbf{x}), \quad \mathbf{x} \in \Omega, \\ \mathcal{B}_k[u(\mathbf{x})] &= g_k(\mathbf{x}), \quad \mathbf{x} \in \Gamma_k \subset \partial\Omega, \quad k = 1, 2, \dots, n_b\end{aligned}\quad (1)$$

where, $\mathcal{D}$, $\mathcal{B}_k$ denote arbitrary differential and boundary/initial operators, respectively. The term refers to the source term or forcing function in the PDE. The functions $g_k(\mathbf{x})$ refer to a set of boundary/initial functions, α is a set of parameter vectors or coefficients, and u is the solution of the differential equation given input $\mathbf{x}$ in the solution domain Ω with boundary.

PINNs [67] provide a framework to embed governing physical laws directly into NN architectures. The PINN loss function combines terms representing governing equations and initial/boundary conditions designed to minimize deviations from physical laws. More formally, the PINN loss function can be defined as:

$$\begin{aligned}\mathcal{L}(\theta) &= \arg \min_{\theta} \sum_{k=1}^n \lambda_k \mathcal{L}_k(\theta) \\ &= \arg \min_{\theta} \left(\lambda_1 \mathcal{L}_1(D[u_{\theta}(\mathbf{x}); \alpha] - f(\mathbf{x})) + \sum_{k=2}^{n_b} \lambda_k \mathcal{L}_k(\mathcal{B}_k[u_{\theta}(\mathbf{x})] - g_k(\mathbf{x})) \right),\end{aligned}\quad (2)$$

where, n_b denotes the total number of loss terms,

$\mathcal{L}_k$ are the individual loss functions for each constraint with weighting coefficients λ_k , and θ represents the trainable parameters.

While the initial and boundary conditions guarantee a unique solution by satisfies predefined constraints at specific spatial or temporal locations, the physics loss acts as a regularizer, guiding the network to learn physically consistent solutions and preventing overfitting to sparse or noisy data. The balance between these loss components is crucial for achieving accurate solutions, as improper weighting can lead to underfitting the initial/boundary conditions or failing to capture the system dynamics correctly. The present work adopts soft imposition of the boundary and initial conditions using fixed weights with the best empirical results. However, various weighting methods have been proposed in the literature using different criteria, such as gradient information [68–70], loss-term history statistics [71, 72], training dataset information [73–75], or hard imposition of initial and boundary conditions [76, 77], to name a few.

3.2. CV QNN

In photonic quantum computing, CV computation addresses quantum states with continuous degrees of freedom, such as the position (x) and momentum (p) quadratures of the electromagnetic field. Within this framework, qumodes represent the fundamental units of the CV quantum systems, analogous to qubits in DV systems. Unlike qubits, qumodes encode information in the continuous degrees of freedom of electromagnetic field modes, rather than discrete two-level states. The computational space is, in principle, unbounded, involving infinite-dimensional Hilbert spaces with continuous spectra. To make these simulations computationally feasible, a cutoff dimension is employed to limit the number of basis states used for approximation. The computational state space is defined as $r = m^n$, with n qumodes and cutoff dimension m . For example, with three qumodes and a cutoff dimension of two, the state space is eight-dimensional. Ideally, higher cutoff dimensions (up to a certain threshold) yield more precise results but at the cost of increased computational complexity.

In this work, we adhere to the affine transformation introduced in [44]. The proposed circuit ansatz consists of a sequence of operations that manipulate quantum states in the phase space. Each layer of the CV-circuit QNN includes the following key components:

1. **Interferometers** ($U_1(\psi_1, \phi_1)$ and $U_2(\psi_2, \phi_2)$): Linear optical interferometers perform orthogonal transformations, represented by symplectic matrices. These transformations adjust the state by applying rotation and mixing between modes, corresponding to beam splitting and phase-shifting operations.
2. **Squeezing gates** ($S(r, \phi_s)$): These gates scale the position and momentum quadratures independently, introducing anisotropic adjustments to the phase space. Encoding sharp gradients using squeezing gates is particularly appealing for PDE solvers, as it allows fine-tuning of local characteristics [17].

3. **Displacement gates** ($D(\alpha, \phi_d)$): These gates shift the state in phase space by adding a displacement vector α , translating the state globally without altering its internal symplectic structure.
4. **Non-Gaussian gate** ($\Phi(\lambda)$): To introduce nonlinearity, a non-Gaussian operation, such as a cubic phase, Kerr, and cross-Kerr gates, is applied. This step enables the NN to approximate nonlinear functions and enhances the ability to solve complex, nontrivial PDEs.

These components together mimic the classical affine transformation such that:

$$L(|x\rangle) = |\Phi(Mx + b)\rangle = \Phi(D \circ U_2 \circ S \circ U_1), \quad (3)$$

where, M represents the combined symplectic matrix of the interferometers and squeezing gates, x is the input vector in the phase space, b is the displacement vector with nonlinear transformation Φ .

In addition to the building blocks above, CV-Circuit QNNs offer flexibility in measurement schemes and parameterization strategies. Measurement can be performed in either the number basis, which projects the state onto discrete photon-number states, or the quadrature basis, which yields continuous-valued outcomes corresponding to position or momentum observables. The parameterization of unitary operations may either be full (including both magnitude and phase) or restricted to magnitude-only (by setting phase parameters ϕ to zero), which can improve trainability and mitigate optimization challenges at the cost of reduced expressivity.

Although such a model exhibits intriguing theoretical properties, their practical implementation in real-world problem solving seems to suffer from fundamental stability issues. This is mainly due to their inherent sensitivity and reliance on continuous degrees of freedom, which complicate optimization compared to DV-based approaches (discussed in detail in section 3.3). This sensitivity stems from several factors, including the precision required for state preparation, noise from decoherence, and the accumulation of errors in VQC. Furthermore, the transformations involved in CV quantum circuits are prone to numerical instabilities, particularly when optimized using gradient-based methods. Although foundational studies about the barren plateau problem, such as [78–80], primarily address DV circuit systems, the analysis indicates that similar issues can arise in CV quantum systems. In particular, the infinite-dimensional Hilbert space and the continuous nature of parameterization can exacerbate the barren-plateau effects.

3.3. DV QNN

DV-Circuit quantum models do not inherently provide affine transformations as a core feature like CV-based QNNs discussed in section 3.2. However, they can achieve affine-like behavior through parameterized single-qubit rotations, controlled operations, and phase shifts for amplitude adjustments. Therefore, the affine transform of equation (3) can be reinterpreted to align with their architecture such that:

1. **$Mx + b$** : can be achieved through a combination of parameterized quantum gates:
 - U_1, U_2 : A unitary operator that can be represented by parameterized single-qubit rotations, such as $RX(\theta)$, $RY(\theta)$, or $RZ(\theta)$.
 - S : A sequence of controlled gates (e.g. CNOT or CZ) responsible for introducing entanglement, which enables the representation of correlated variables and interactions.
 - D : Data encoding circuits that map classical data into quantum states through techniques such as basis embedding, amplitude embedding, or angle embedding.
2. **Non-linear activation (Φ)**: DV-Circuit QNNs lack intrinsic quantum nonlinearity due to the linearity of quantum mechanics. However, classical measurements or a combination of classical measurements and non-linear activation functions (such as Tanh) can be applied to the output between quantum layers [81, 82].

4. QCPINN

Figure 1 illustrates a high-level overview of our proposed QCPINN architecture. The preprocessor consists of a two-layer classical NN. The first layer performs a linear transformation to modify the input dimension, followed by a nonlinear activation function to introduce classical non-linearity. The second layer subsequently maps the transformed inputs to the QNN layer (or classical NN layer used for comparison). The postprocessor layer mirrors the structure of the preprocessor, maintaining consistent dimensions and ensuring a symmetric transformation between classical and quantum representations. This design provides flexibility in encoding classical inputs to quantum inputs and decoding

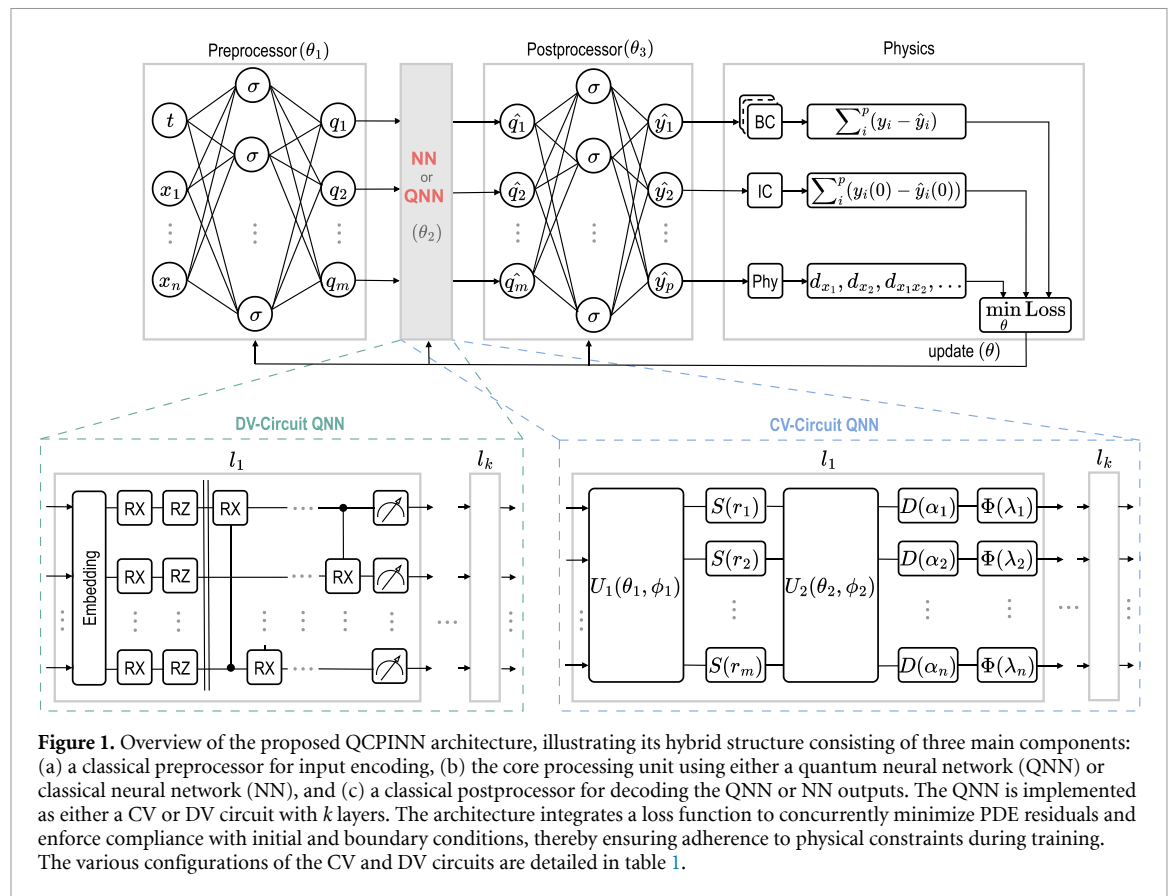


Table 1. Various configurations of the QCPINN, along with the list of PDEs used for testing. Each configuration was evaluated against the Helmholtz and lid-driven cavity cases, with the top-performing setups subsequently selected for further assessment on additional scenarios, including Wave, Klein–Gordon, and Convection–diffusion problems.

QNN	Configurations		PDEs
	Type	Options	
CV-Circuit	Measurement schemes	Number, Quadrature	(tested on all configurations)
	Nonlinear operations	Kerr, Cubic phase, Cross-kerr	Helmholtz, Lid-driven cavity
	Parameterization	Phase-free (magnitude), Full (magnitude and phase) (see algorithm 1)	(tested on selected configurations)
DV-Circuit	Embeddings	Amplitude, Angle	Wave, Klein–Gordon,
	Topology	Alternate, Cascade, Cross-mesh, Layered (see figure 2 and table 2)	Convection–diffusion

quantum outputs back to classical outputs. Additionally, it manages input-output dimensions and regulates quantum feature representations, all while ensuring the higher-order differentiability necessary for residual loss in the proposed QCPINN model.

4.1. CV-Circuit QCPINN

The CV-Circuit QCPINN consists of three primary components: a classical preprocessor network, a CV-Circuit quantum network (the QNN layer of figure 1), and a classical postprocessor network. The quantum network is adapted from [44] and is described in algorithm 1. Each layer of the quantum network executes a sequence of quantum operations. Table 1 presents various configurations of our CV-Circuit studied to identify a suitable parameterization scheme, nonlinear operation, and measurement approach. For measurement schemes, we evaluated both number and quadrature. For parameterization, we examined phase-free using only magnitude parameters and full parameterization using both magnitude and phase parameters. For nonlinear operations, we explored Kerr nonlinearity, cubic phase gates,

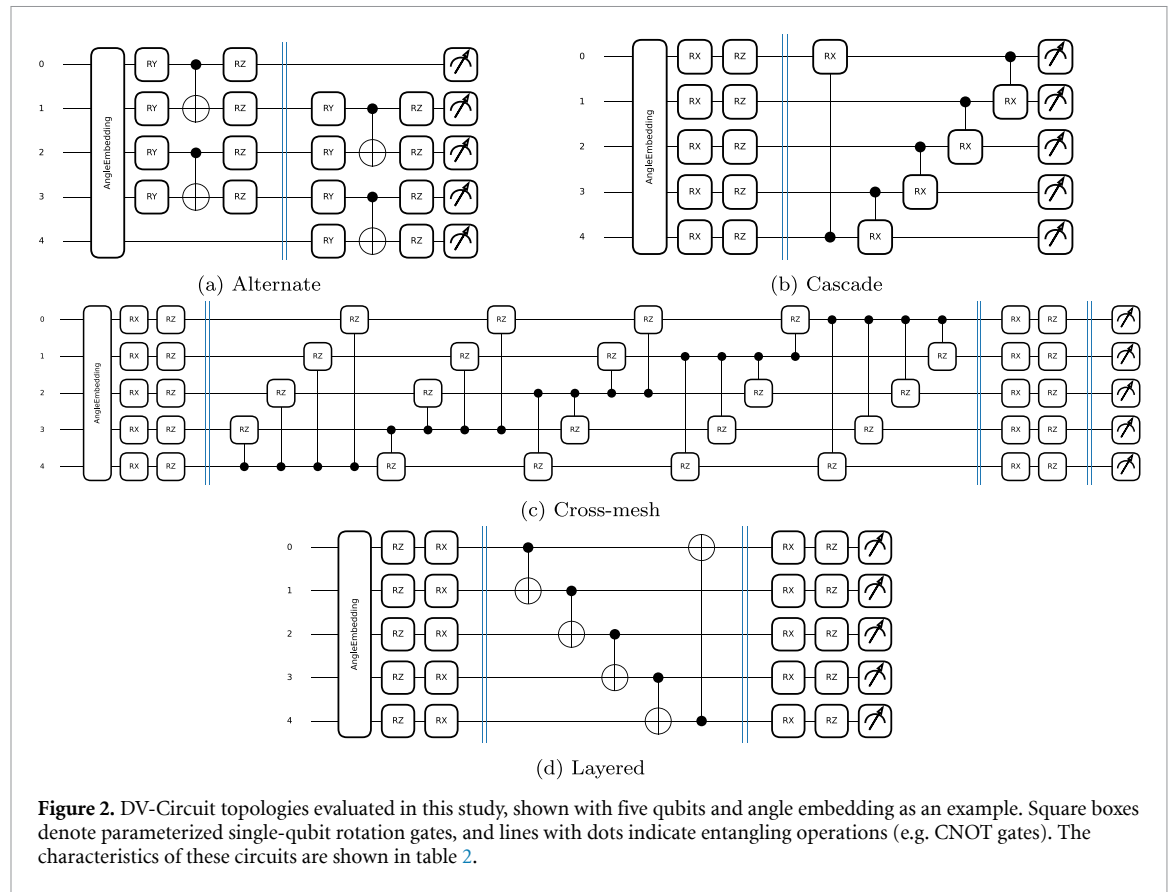


Figure 2. DV-Circuit topologies evaluated in this study, shown with five qubits and angle embedding as an example. Square boxes denote parameterized single-qubit rotation gates, and lines with dots indicate entangling operations (e.g. CNOT gates). The characteristics of these circuits are shown in table 2.

Table 2. Characteristics of the topologies used in DV-Circuits in figure 2, where n is the number of qubits, and L is the number of layers.

Topology	Circuit depth	Circuit connectivity	Number of parameters	Number of two-qubit gates
Alternate	$6L$	Nearest-neighbor	$4(n-1)L$	$(n-1)L$
Cascade	$(n+2)L$	Ring topology	$3nL$	nL
Cross-mesh	$(n^2 - n + 4)L$	All-to-all	$(n^2 + 4n)L$	$(n^2 - n)L$
Layered	$6L$	Nearest-neighbor	$4nL$	$(n-1)L$

and cross-Kerr interactions. Additionally, the quantum state is measured, typically in either the quadrature basis $\langle \hat{q}_i \rangle$ or the Fock number basis $\langle \hat{n}_i \rangle$, to yield classical outputs for the postprocessing layers. Please refer to section 3.2 for additional details.

Structuring the network in this way results in a circuit depth of $(2n+3)L$, while the number of trainable quantum parameters increases to $\mathcal{O}(n^2L)$, with n qumodes and L number of layers.

4.2. DV-Circuit QCPINN

Similar to the QCPINN (CV-Circuit) discussed in section 4.1, the DV-Circuit QCPINN consists of three primary components: a classical preprocessor network, a quantum network (the QNN layer of figure 1), and a classical postprocessor network. The quantum network is a custom DV-Circuit, which executes quantum operations on the preprocessed data. Table 1 presents various configurations of our DV-Circuit studied to identify a suitable embedding scheme and topology. Figure 2 illustrates four proposed ansätze named after their topology: Alternate, Cascade, Cross-mesh, and Layered. These circuits are adapted from literature, including [58, 83–85]. All square boxes in the figure denote parameterized single-qubit or controlled-rotation gates (RX, RY, RZ, CRX, CRZ), which are optimized during training.

Table 2 presents features of the DV-Circuits across the key descriptors, circuit depth, connectivity, number of parameters, and the number of two-qubit gates. Here, circuit depth refers to the longest computational path in the circuit from the data input to the output, which represents the maximum number of logically dependent gates executed on a single qubit. This metric is critical as deeper circuits require more computational resources and may incur numerical errors that can degrade gradient quality during NN optimization. The chosen circuits agree with the affine transformation discussed in section 3 and

adhere to the hardware-efficient ansatz [86] defined as:

$$U(\psi) = \prod_k U_k(\psi_k) W_k, \quad (4)$$

where, each layer k alternates between single-qubit parameterized rotations, $U_k(\psi_k)$, and an entangling operation, W_k .

The **Cross-mesh** ansatz implements an all-to-all connectivity scheme, facilitating extensive interaction between qubits, which enhances its ability to explore the Hilbert space. The circuit depth scales as $\mathcal{O}(n^2 - n + 4)L$, while the number of trainable parameters is $(n^2 + 4n)L$. The use of global entanglement fosters strong correlations between qubits; however, it results in larger circuit depth, resource demands, and training difficulties [80]. This circuit's total number of two-qubit gates is also significantly higher than in other ansätze.

The **Cascade** ansatz balances expressivity and hardware efficiency by employing a ring topology for qubit connectivity. This design facilitates efficient entanglement while keeping the circuit depth manageable at $(n + 2)L$. The circuit contains $3nL$ trainable parameters, making it more efficient than Cross-mesh while maintaining reasonable expressivity. Incorporating controlled rotation gates (CRX) improves entanglement compared to CNOT gates and increases the circuit's capacity to explore the Hilbert space. Furthermore, the requirement for two-qubit gates is fewer than that of Cross-mesh, thus lowering the overall computational cost.

The **Layered** ansatz features a structured design that includes alternating rotation and entangling layers. Each qubit undergoes parameterized single-qubit rotations, typically utilizing RX and RZ gates. Entanglement is achieved through nearest-neighbor CNOT gates. Compared to the Cross-mesh's all-to-all connectivity or the Cascade's ring topology, the Layered architecture represents a compromise between expressivity and hardware efficiency.

Among the evaluated DV topologies, the Alternate ansatz employs alternating layers of parameterized single-qubit rotations followed by nearest-neighbor CNOT gates, ensuring effective entanglement while maintaining a manageable circuit depth of $6L$ and $4(n - 1)L$ trainable parameters. Although we did not compute entanglement entropy explicitly, this layout is known to generate strong bipartite and multipartite entanglement [12, 80, 84].

Compared to the QNN models discussed above, we can see that the classical feed-forward NNs exhibit a more flexible scaling pattern, with trainable parameters scale as $\mathcal{O}(h_i h_{i+1})$, where h_i and h_{i+1} denote the number of neurons in two consecutive layers l_i and l_{i+1} with independent selection of the neurons. As a result, while classical models depend on deep architectures and larger parameter counts, quantum networks achieve expressivity by utilizing intrinsic quantum correlations and exponential scaling of the Hilbert space. Consequently, incorporating a QNN between two consecutive classical layers, like the structure shown figure 1, with number of qubits much less than the number of neurons can reduce the number of parameters from, say, $\mathcal{O}(h_i h_{i+1})$ to $\mathcal{O}(h_i n_i + q_{\text{params}} + n_i h_{i+1})$ where n_i is the number of qubits and q_{params} represents the quantum circuit parameters.

5. Implementation and training

5.1. CV-Circuit QCPINN

We implemented the preprocessor and the postprocessor (as in figure 1) with one hidden layer with 50 neurons and Tanh activation function. We used the PennyLane framework [64] with the Strawberry Fields Fock backend [87] for the CV-Circuit QCPINN implementation. Each CV-Circuit layer was configured for two qumodes with a cutoff dimension of 20. Quantum parameters were initialized using controlled random distributions, with small values for active parameters (e.g. r , d) to ensure numerical stability. Classical NN layers were initialized using Xavier initialization [88]. During training, either position quadrature (X) or number operator expectation measurements are performed on each qumode to extract continuous-valued features from the quantum state.

As CV-circuits rely on continuous degrees of freedom, their operations are inherently more sensitive to parameter scaling than those in DV systems. This sensitivity often manifests as instability during training, particularly when parameters grow large or when nonlinear transformations amplify fluctuations in the quantum state. For instance, squeezing operations are highly sensitive to large values, as excessive squeezing amplifies quadrature fluctuations, which increase photon number variance and truncation errors in Fock-space representations [89]. Similarly, displacement operations are constrained to prevent quantum states from exceeding computational cutoffs, as this can introduce simulation errors [90]. Likewise, the nonlinear operations such as the Kerr gate can cause phase shifts that scale quadratically with photon numbers, making them highly sensitive to large values [91].

Therefore, we used a learning rate of 1.0×10^{-4} to address the model's sensitivity to gradient updates. A ReduceLROnPlateau scheduler was implemented with a patience of 20 epochs, decreasing the learning rate by a factor of 0.5 when no improvement was observed, down to a minimum of 1.0×10^{-6} . Additionally, we applied gradient clipping to prevent exploding gradients, restricting their norm to a maximum of 0.1. We also scaled the training parameters by small factors to further stabilize the CV transformations. While these settings can alleviate the instability and prevent divergence in quantum state evolution, they can also introduce the vanishing gradient problem [88].

5.2. DV-Circuit QCPINN

Similar to the CV-Circuit QCPINN, the preprocessor and the postprocessor consist of one hidden layer with 50 neurons and the Tanh activation function. We used Adam optimizer with a learning rate of 0.005 to train the DV-Circuit QCPINN. A learning rate scheduler, ReduceLROnPlateau, is employed to adjust the learning rate when the loss plateaus dynamically. Specifically, the scheduler reduces the learning rate by a factor of 0.9 if no improvement is observed for 1000 consecutive epochs, promoting stability and convergence. Furthermore, the training function applies gradient clipping to prevent exploding gradients, clipping them according to their norm to ensure they do not exceed a unity value. During training, Pauli-Z expectation measurements are performed on each qubit to extract features from the quantum state.

In all DV-Circuit QCPINN experiments, we set the number of shots to None to enable analytic gradient computation through backpropagation with PennyLane. This configuration activates the deterministic simulation mode, allowing continuous gradients to flow through the quantum circuit and enabling differentiation with backpropagation. Using a finite number of shots would introduce stochastic sampling noise, which would disrupt compatibility with backpropagation and require alternative differentiation methods, such as the parameter-shift rule, that are significantly more computationally expensive.

5.3. Classical PINN models

The baseline PINN models consist of three main components: a preprocessing network, a NN, and a postprocessing network (as shown in figure 1). The preprocessor and the postprocessor contain one hidden layer with 50 neurons and the Tanh activation function. The NN has two different configurations:

1. Model-1 includes two fully connected layers, each containing 50 neurons with a Tanh activation function between layers, and
2. Model-2 retains only the preprocessing and postprocessing networks, entirely omitting the NN to simplify the model's structure.

These two models represent different levels of complexity, with model-1 having more layers and, consequently, a higher number of trainable parameters than model-2. We use these baseline PINN models to evaluate and compare the performance of the proposed hybrid models in figure 1.

5.4. PDEs

We implemented five distinct PDEs, which include the Helmholtz, the time-dependent 2D lid-driven cavity, the 1D Wave, the Klein–Gordon, and a Convection–diffusion problem. The mathematical formulation, boundary conditions, and corresponding loss function designs for each PDE are detailed in appendix A.

5.5. Experiment setup

We used PyTorch's automatic differentiation to compute the gradients for both classical and quantum parameters, and the Adam optimizer. The training was carried out with a batch size of 64 and a maximum of 20 000 epochs. For the cavity problem, the entire training dataset was generated using the Sobol sequence; in other cases, random sampling was sufficient throughout the training process. Training settings, including the optimizer and batch size, were kept consistent in QCPINN models to allow for a fair comparison with the baseline PINN. While altering configurations, such as increasing iterations or adjusting the batch size, might improve performance, the aim is to evaluate the models under identical conditions. Given the near-infinite hyperparameter space, this controlled evaluation ensures a fair assessment of the relative strengths and limitations of each approach. In our initial implementation, we observed that the experiments on the CPU were faster than those on the GPU. This is likely due to our study's relatively small batch size, circuit complexity, and limited number of qubits/qumodes: five qubits for DV-Circuit models and two qumodes for CV-based models. Moreover, the quantum backends

Table 3. Comparing the results of the best configuration of DV-Circuit QCPINN with PINN. Reported metrics include N_p : the number of trainable parameters; L_2 : relative error (in %), and the mean $\pm$ std across 10 independent runs; L_{final} : final loss; T : time per iteration (in sec); M : peak memory consumption (in MB). For the cavity problem, u and v : velocity and p : pressure, and for other cases, u : reference solution and f : source term. For results with additional configurations, see tables 5–7 in the appendix.

PDEs	Model		N_p		L_2	$\mathcal{L}_{\text{final}}$	T	M
Helmholtz equation (5)	QCPINN	Angle-Cascade	771	u	6.69 ± 1.51	4.52 ± 1.86	0.17 ± 0.01	420.0 ± 0.0
				f	2.86 ± 0.54			
	PINN	Model-2	2751	u	12.12 ± 7.17	5.04 ± 3.12	0.002 ± 0.01	340.0 ± 0.0
				f	3.23 ± 1.19			
Cavity equation (6)	QCPINN	Angle-Cascade	923	u	25.73 ± 2.49	0.10 ± 0.02	0.36 ± 0.01	540.0 ± 0.0
				v	38.00 ± 3.76			
				p	42.65 ± 8.69			
	PINN	Model-1	8003	u	21.61 ± 6.68	0.06 ± 0.02	0.01 ± 0.0	440.0 ± 0.0
				v	34.24 ± 2.76			
				p	34.82 ± 5.25			
Wave equation (7)	QCPINN	Angle-Cross-mesh	796	u	12.44 ± 4.42	0.09 ± 0.03	0.38 ± 0.08	469.0 ± 0.0
	PINN	Model-2	2751	u	11.13 ± 1.78	0.05 ± 0.01	0.02 ± 0.02	351 ± 0.00
Klein–Gordon equation (8)	QCPINN	Angle-Cascade	771	u	4.24 ± 1.88	0.27 ± 0.10	0.25 ± 0.03	447.0 ± 0.0
				f	1.72 ± 0.50			
	PINN	Model-2	2751	u	4.48 ± 1.72	0.34 ± 0.13	0.02 ± 0.02	350.0 ± 0.0
				f	1.79 ± 0.47			
Convection–diffusion equation (9)	QCPINN	Angle-Cross-mesh	821	u	4.54 ± 2.17	0.001 ± 0.001	0.42 ± 0.11	468.0 ± 0.0
				f	2.63 ± 1.61			
	PINN	Model-1	7901	u	12.62 ± 6.05	0.02 ± 0.02	0.01 ± 0.0	348.0 ± 0.0
				f	5.57 ± 3.05			

employed in our experiments are not fully optimized for GPU efficiency. Considering these limitations, we have conducted our experiments on a CPU system with 6148 CPUs (each operating at 2.40 GHz) and 192 GB of RAM.

6. Results

We first investigated the performance of the CV-Circuit QCPINN models across twelve distinct configurations, encompassing two measurement schemes, three nonlinear operations, and two parameterization strategies (as summarized in table 1). These configurations were evaluated on the Helmholtz and lid-driven cavity problems to assess their stability and convergence characteristics. However, the training behavior of the CV-Circuit models was found to be highly unstable, regardless of the configuration tested. Results are discussed in appendix B in detail. In particular, table 4 and figure 9 report the corresponding solution accuracy and training convergence for two representative configurations.

Following the CV-Circuit analysis, we systematically evaluated the DV-Circuit QCPINN models across a set of configurations comprising two embedding schemes and four circuit topologies. These quantum architectures were benchmarked against baseline PINN models discussed in section 5.3 to assess their relative performance in terms of accuracy, convergence, parameter efficiency, time, and memory. We report the results of the best-performing configurations, while the complete set of results for all DV-Circuit configurations is provided in appendix C for reference.

6.1. Helmholtz equation

Table 3 presents the performance of the best-performing DV-Circuit QCPINN for solving the Helmholtz equation (5) and comparing the results with the best-performing PINN. The QCPINN (Angle-Cascade) with 771 parameters outperforms the PINN (Model-2) with 2751 parameters, achieving a parameter reduction of $\sim 72\%$. This results in a $\sim 45\%$ lower relative L_2 error for the solution field u (6.69% vs 12.12%) and 11% lower error for the force term f (2.86% vs 3.23%) at the expense of time and memory.

Figure 3 compares the training loss history of the best-performing models. The curve shows the mean loss over ten independent runs, with shaded regions indicating the standard deviation. Both models exhibit stable convergence, with the QCPINN achieving slightly lower final loss and reduced variance,

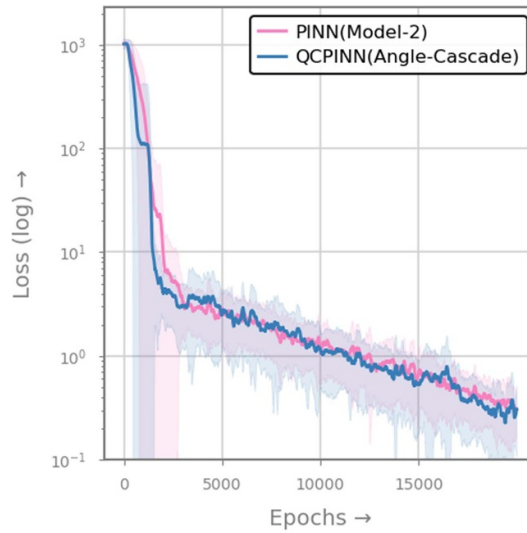


Figure 3. Comparison of the loss history of the best models of DV-Circuit QCPINN (Angle-Cascade) with PINN (Model-2) for solving the Helmholtz equation. For loss history of additional models, see figure 10 in the appendix.

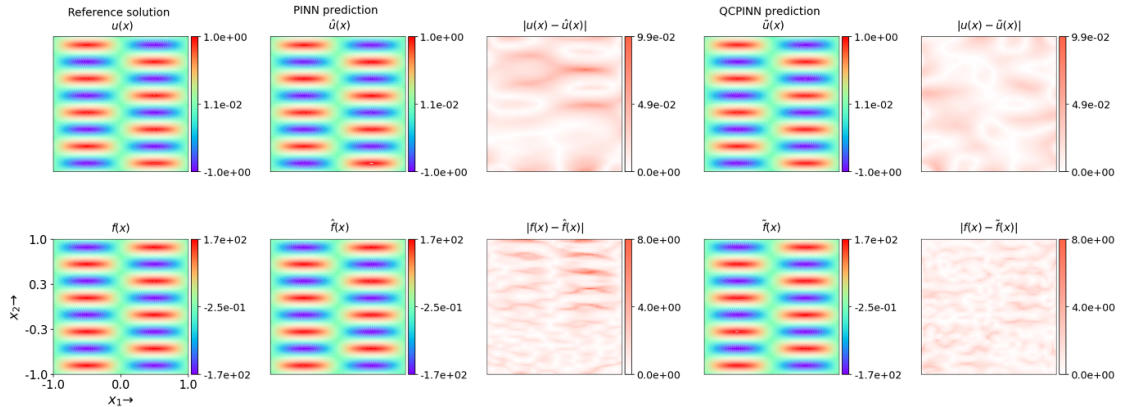


Figure 4. Comparison between the reference solution and model predictions for the Helmholtz equation. This includes the best results from our DV-Circuit QCPINN (Angle-Cascade) alongside PINN (Model-2). The first row shows the PDE solution u , while the second row shows the force field f .

indicating more consistent training performance across runs despite using significantly fewer trainable parameters.

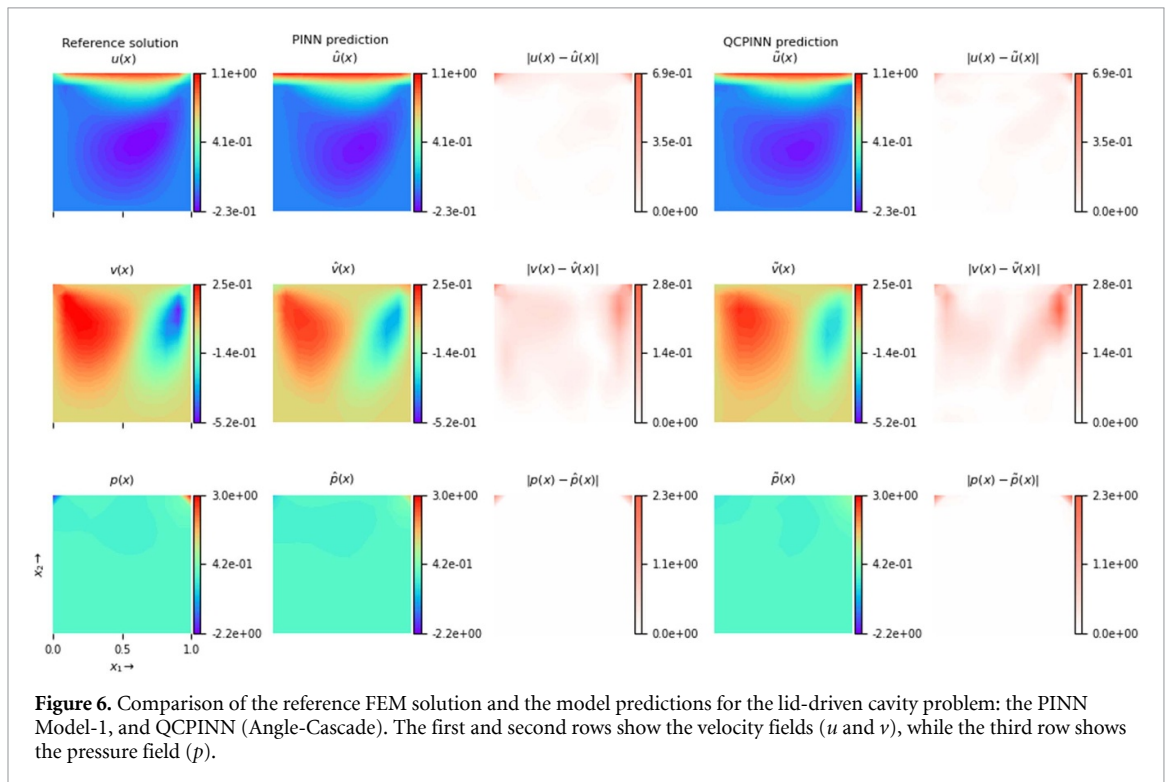
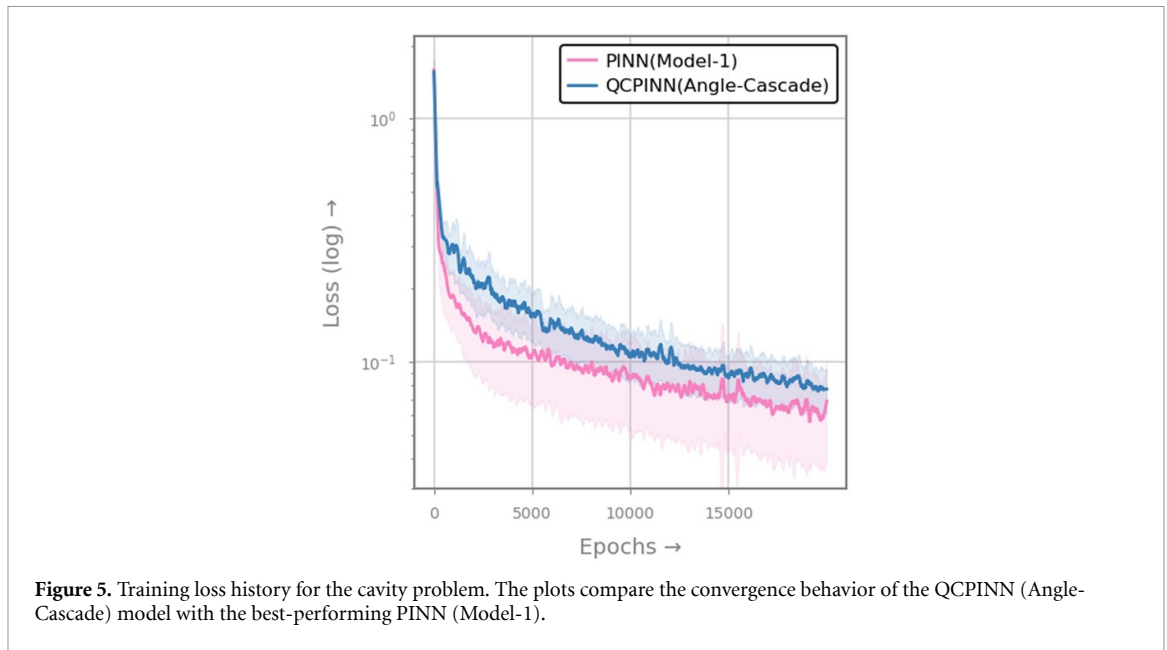
Figure 4 presents the reference and predicted solutions, along with the corresponding absolute error distributions. Both QCPINN and PINN models accurately capture the overall solution pattern with visible deviations near the domain boundaries and regions with sharp local gradients.

6.2. Time-dependent 2D lid-driven cavity equation

Table 3 shows that the lid-driven cavity problem presents a balanced trade-off between parameter efficiency and predictive accuracy. The QCPINN (Angle-Cascade) achieves an $\sim 88\%$ reduction in trainable parameters compared to PINN Model-1 (923 vs 8003), demonstrating substantial model compactness. Although this reduction is accompanied by modest increases in the relative errors for the velocity components u , v , and p ($\sim 19\%$, $\sim 11\%$ and $\sim 23\%$, respectively).

Figure 5 compares the training loss convergence. Both models achieve stable convergence with minimal oscillations, indicating well-behaved optimization. The QCPINN reaches a slightly higher final loss value (0.10 vs 0.06) as shown in table 3 but follows a smoother convergence trajectory, suggesting stable learning dynamics even with significantly fewer parameters.

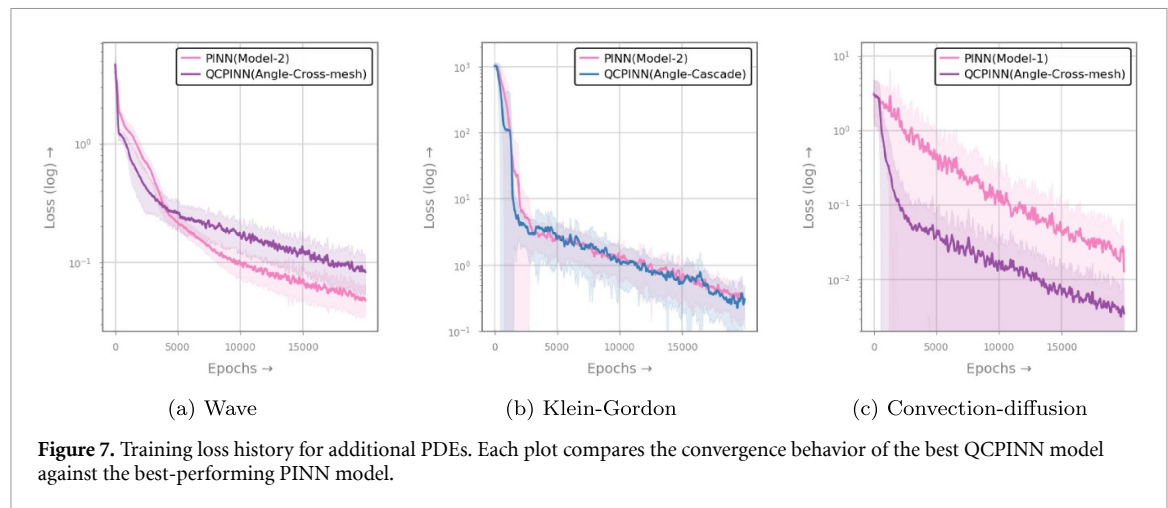
Figure 6 visualizes the predicted velocity and pressure fields along with their absolute error maps. Both models reproduce the dominant flow structure, including the central primary vortex and secondary corner eddies, indicating that they effectively capture the overall flow dynamics. However, significant differences emerge in the pressure field. The largest pressure errors for both models occur near the upper



moving lid and corner regions, where sharp velocity gradients and strong shear layers lead to steep pressure variations. These regions are inherently challenging for PINN-based models, as the physics-informed loss involves higher-order derivatives that amplify numerical sensitivity in zones with discontinuous boundary forcing.

6.3. Additional PDEs

Results from all three additional PDEs (Wave, Klein–Gordon, and Convection–diffusion) consistently show that Angle embedding significantly outperforms Amplitude embedding across all examined cases. Additionally, the Cascade and Cross-mesh topologies demonstrated superior performance compared to the other topologies tested. Building on these findings, we conducted further tests on these PDEs using the most effective configurations. The results are presented in table 3, while additional results can be found in table 7 in the appendix.



In the Wave equation, the Angle-Cross-mesh QCPINN attains a $\sim 71.1\%$ reduction in parameters (796 vs. 2751) with a slightly higher, $\sim 11.8\%$ increase, in relative L_2 error for u (12.44% vs. 11.13%).

For the Klein-Gordon equation, the Angle-Cascade QCPINN yields $\sim 72.0\%$, parameter savings (771 vs. 2751) while slightly improving accuracy: the mean L_2 error for u decreases from 4.48% to 4.24%, $\sim 5.4\%$ relative improvement, and the error for f decreases marginally, $\sim 3.9\%$. These differences are small and lie within the reported standard deviations, indicating comparable performance with fewer parameters.

In the Convection-diffusion equation, the Angle-Cascade QCPINN demonstrates both large parameter efficiency (821 vs. 7901 parameters, $\sim 89.6\%$ reduction) and substantially better accuracy: the mean L_2 error for u falls from 12.62% to 4.54% ($\sim 64.0\%$ relative improvement), and the error for f decreases from 5.57% to 2.63% ($\sim 52.8\%$ relative improvement). As shown in table 7, all QCPINN models perform well on this problem. This case shows a clear advantage of the QCPINN architecture in capturing sharp localized features typical of convection-dominated flows.

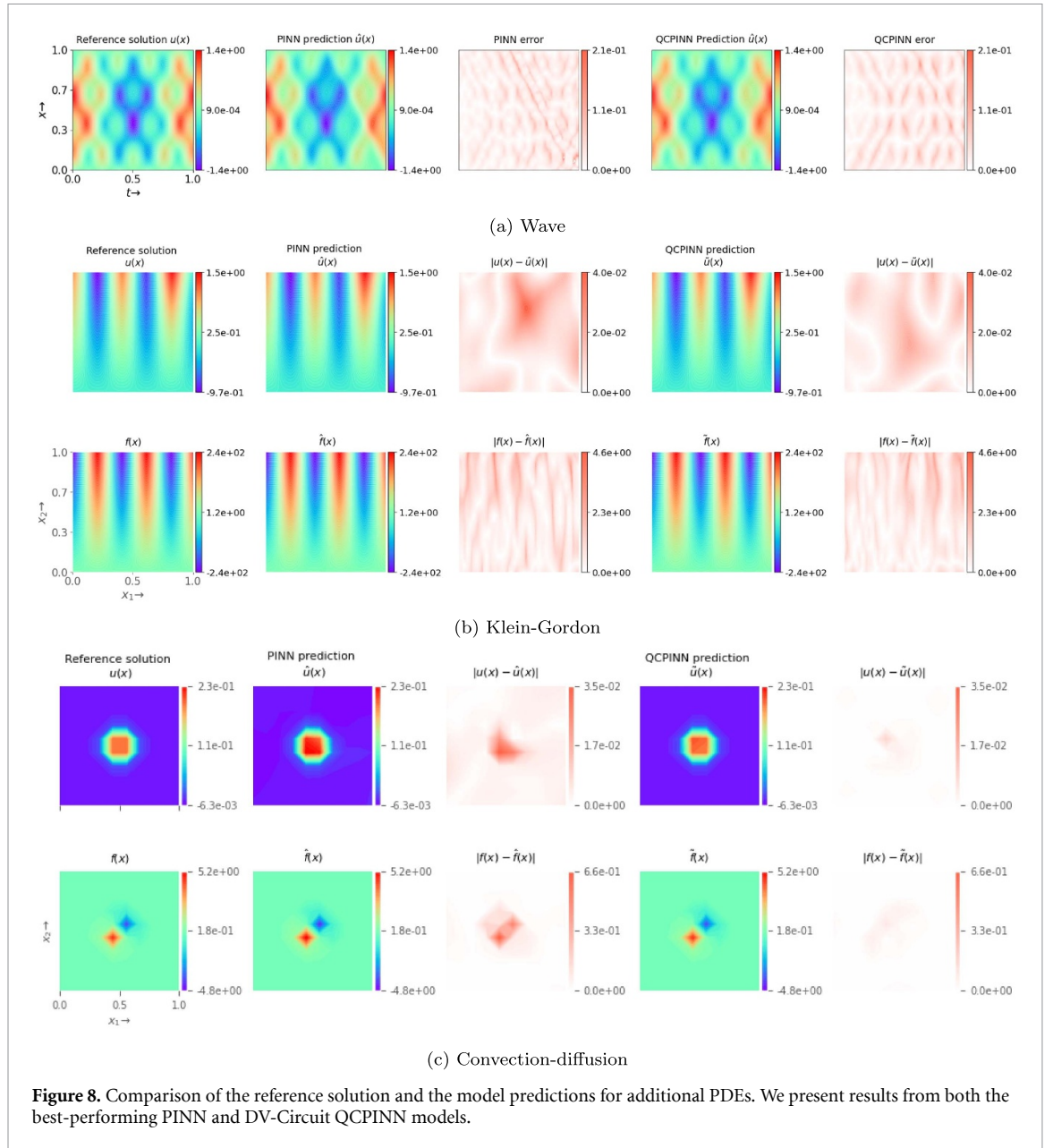
Figure 7 compares the training loss histories of these models across the additional PDEs. For the Wave equation, figure 7(a) shows that the QCPINN achieves faster initial convergence, reaching lower loss values within the first 4000 epochs before the baseline PINN (Model-2) begins to converge more rapidly in the later stages.

For the Klein-Gordon equation, figure 7(b) demonstrates that both models follow nearly similar convergence trajectories. In contrast, for the Convection-diffusion equation, figure 7(c) highlights a clear advantage for the QCPINN, which exhibits a steadily decreasing loss throughout training and attains a final loss approximately an order of magnitude lower than the classical model. This result aligns with table 3, where the QCPINN achieved lower relative L_2 errors, confirming its better capability in handling convection-dominated dynamics with sharp localized gradients.

Figure 8 shows a comparison of the reference solutions and the predictions from both the best-performing PINN and the DV-Circuit QCPINN models. The plots show that both models effectively capture the overall distribution of the solution, while most errors occur in regions of high curvature or sharp transitions. For the Wave equation, the QCPINN closely follows the propagation patterns of the reference field, reproducing the oscillatory structure with only minor deviations near steep gradients. In the Klein-Gordon case, both models capture the coupled oscillatory behavior of the solution, with the QCPINN maintaining stable amplitude and phase characteristics despite its reduced parameter count. The advantages of the QCPINN are particularly pronounced in the Convection-diffusion equation, as it better preserves sharp, localized features typical of convection-dominated dynamics compared to the PINN solution.

6.4. Memory and time consideration

Across all experiments, the QCPINN models demonstrate a significant increase in computational time per iteration, exhibiting a 15 to 40-fold increase compared to PINN approaches. This discrepancy highlights the computational overhead associated with simulating quantum circuits on classical hardware. The memory consumption remains relatively consistent between quantum and classical approaches, ranging from 420 to 540 MB for QCPINN models, compared to 340 to 440 MB for conventional PINN models. This observation suggests that the hybrid architecture does not impose additional memory requirements. While the computational overhead is a characteristic of the simulator, we anticipate that



substantial advantages will become evident as quantum hardware becomes more accessible and practical for implementation.

7. Discussion

Circuit topology and entanglement in QCPINN. Both the Cross-mesh and Cascade topologies share key structural properties that contribute to their good performance in the QCPINN models. Each employs parameterized controlled-rotation (CRX or CRZ) gates that introduce continuously tunable entanglement between qubits, supporting smoother gradient propagation and mitigating barren-plateau effects during hybrid training (as in tables 5–7 and figures 10–12). While the Cascade design distributes entanglement efficiently through a ring topology at shallow depth, the Cross-mesh circuit extends this advantage to an all-to-all connectivity pattern. In contrast, the Alternate and Layered circuits rely on fixed CNOT gates, generating discrete, local entangling operations that offer limited tunability and are more prone to optimization plateaus, as reflected in their slower convergence and higher residual losses for the Wave equation (as in table 7 and figure 12(a)).

Overall, the comparison indicates that hybrid QCPINN architectures benefit most from hardware-efficient ansätze with continuous entanglement control, such as the Angle-Cascade and Angle-Cross-mesh circuits. These configurations offer a favorable balance between expressivity, trainability, and hardware feasibility, making them highly suitable for near-term quantum solvers of PDEs.

Angle vs amplitude embedding. Results consistently demonstrate that Angle embedding achieves better performance compared to Amplitude embedding across all tested configurations (as in tables 5 and 6). This advantage arises from both theoretical and practical considerations. First, Amplitude embedding represents the data as quantum state amplitudes that must satisfy the normalization constraint $\sum_i |a_i|^2 = 1$, coupling all input features globally and introducing nonlinear dependencies that complicate optimization. Second, it scales poorly with input dimensionality, requiring 2^n amplitudes to encode n features, which often require zero-padding (or, in our case, we can also use a classical preprocessor with 2^n outputs, which increases expressivity by introducing new trainable parameters but does not yield better results, probably due to global coupling through normalization constraint and degraded gradient flow during backpropagation).

In contrast, Angle embedding directly maps each classical feature to a rotation angle on a single qubit through parameterized gates such as $RX(\theta)$, allowing local and differentiable embeddings with straightforward gradient computation via automatic differentiation. Hence, it provides hardware-efficient, normalization-free feature mapping in which each input independently controls one qubit rotation, yielding stable gradients, better generalization, and faster convergence. Empirically, our angle-encoded circuits (e.g., Angle-Cascade) achieved 45%–64% lower relative L_2 errors than their amplitude-encoded counterparts across the Helmholtz and cavity problems (as in tables 5 and 6), confirming that feature-to-rotation mappings are better suited for such hybrid QCPINN architectures.

Feasibility and noise analysis. While all experiments in this study were performed on noise-free simulators, practical implementations on near-term quantum processors will inevitably be affected by shot noise from finite measurements, gate infidelity, and decoherence (T_1/T_2 relaxation). Nevertheless, the feasibility of implementing the proposed QCPINN models on near-term quantum hardware depends on qubit or qumode requirements, circuit depth, entanglement cost, sampling demands, and expected resilience to device noise. For the DV-Circuit QCPINN, the most effective configurations were the Angle-Cascade and Angle-Cross-mesh topologies. Both were implemented with five qubits and a single quantum layer ($L = 1$), as summarized in section 5. The circuit depth and entangling gate counts vary by topology, as detailed in table 2:

- The Angle-Cascade circuit follows a ring topology with moderate expressivity and hardware efficiency. Its circuit depth scales as $(n + 2)L = 7$, using about five entangling gates and roughly 15 trainable parameters. This shallow depth and limited entanglement make it well-suited for current NISQ devices, which typically tolerate 10–20 two-qubit gates before decoherence significantly impacts fidelity [60].
- The Angle-Cross-mesh circuit adopts all-to-all qubit connectivity. Its depth scales as $(n^2 - n + 4)L = 24$ for $n = 5$, with approximately 20 entangling gates and 45 trainable parameters. Despite being deeper, this layout remained tractable.

For hardware-based training, gradient estimation generally requires 2000–5000 shots per iteration to maintain sampling error below 1% in observable expectations [12]. Dominant hardware noise sources include gate infidelity, readout errors, and T_1/T_2 relaxation. The state-of-the-art superconducting quantum processors achieve single-qubit gate fidelities of 99.9% [92, 93] and two-qubit fidelities around 99.3%–99.7% [94, 95]. Thus, a depth-24 circuit retains effective end-to-end fidelity exceeding 90% before readout correction.

Hyperparameter sensitivity of CV-Circuit QCPINN models. Although the CV-Circuit QCPINN models did not achieve the same performance levels as their DV-based counterparts, these results remain significant for understanding the current feasibility limits of CV QNNs in physics-informed learning. The observed training failures primarily arose from vanishing gradients imposed by stability constraints, compounded by the intrinsic challenges of PINN optimization, namely, high-order differential requirements and a highly non-convex residual loss landscape. These factors intensified the barren plateau effect, leading to persistent residual loss oscillations with minimal improvement over thousands of iterations (figure 9). In other words, the conflict between maintaining valid quantum states (favoring small parameter initialization) and achieving sufficient gradient magnitudes restricted trainability. Among all

hyperparameters, the cutoff dimension, learning rate, and initialization scale of active gates (squeezing and displacement amplitudes) exerted the greatest influence on whether training could escape these plateaus.

Importantly, the fact that the measurement scheme and nonlinearity choice (as in table 1) had minimal impact on performance suggests that the bottleneck lies not in CV-Circuit expressivity, which remains theoretically superior due to their infinite-dimensional Hilbert spaces, but in the optimization landscape dominated by barren plateaus and restricted gradient flow. This insight directs future research toward developing optimization strategies specifically designed to mitigate barren plateaus in CV systems (e.g., natural-gradient descent, adaptive parameter-shift rules, layer-wise training, or gradient-free evolutionary methods) rather than further modifying circuit architectures.

Limitations and future work. Although parameter efficiency was significantly improved, overall solution accuracy remained comparable to baseline PINNs, especially for cavity and Wave equations, where there is still room for enhancement. Additionally, the highly non-convex PINN loss landscape still required careful weighting for the loss terms, which were determined through extensive empirical studies rather than exploring quantum circuit advantage to address this challenge. Contrary to theoretical expectations, CV-Circuit implementations exhibited training instabilities and underperformed relative to DV-Circuit variants for the cases considered in this study. Looking ahead, the promise of quantum-enhanced PINNs in practical applications is evident. Future work should include validation of QCPINN models on quantum hardware to evaluate their resilience to noise and sampling errors—both critical for real-world applications. Additionally, developing advanced quantum circuit designs tailored to PDEs and optimizing training stability, especially for CV methods, will further advance the integration of quantum computing and machine learning for solving complex problems across diverse fields.

8. Conclusion

We found that QCPINN models achieve comparable solution accuracy to baseline PINNs while requiring a fraction of trainable parameters, as little as 10%–30% of those needed by their classical counterparts under about the same settings. The experimental evidence supports our hypothesis that hybrid quantum-classical approaches can maintain solution quality while dramatically improving parameter efficiency. Our systematic assessment of various QCPINN configurations across multiple PDEs consistently revealed that DV-Circuit implementations with Angle embedding and either Cascade or Cross-mesh topologies offer the best performance. Angle embedding is superior to Amplitude embedding due to its simplicity and gradient stability, as it maps data to individual qubit rotations without normalization constraints, allowing for stable backpropagation. The Cascade topology balances expressivity and efficiency, providing enough entanglement to capture complex PDE relationships while keeping circuit depth manageable and avoiding barren plateaus. The Cross-mesh topology, on the other hand, leverages all-to-all qubit connectivity to enhance the circuit's expressivity and its ability to represent globally correlated solution features across the spatial domain of PDEs. While this design increases the number of entangling gates and circuit depth relative to the Cascade layout, our experiments show that the added expressivity improves generalization and convergence stability without excessive parameter growth. For the Helmholtz, Klein–Gordon, and Convection–diffusion equations, our QCPINN achieved a significant reduction of around 4%–64% in relative L_2 error across various fields compared to baseline PINN models. In contrast, for the cavity and Wave equations, the L_2 error was slightly higher across various fields compared to the baseline PINN models. CV-Circuits exhibit persistent training instabilities as standard gradient-based optimization methods, such as Adam, face significant challenges with CV-based quantum circuit models due to their sensitivity to parameter initialization and gradient dynamics. The findings in this study can be used as a practical guideline for designing hybrid quantum-classical neural architectures to solve PDEs with improved parameter efficiency and stable convergence. The open-source implementation further enhances reproducibility and enables community engagement.

Data availability statement

The data that support the findings of this study are openly available at the following URL/DOI: <https://github.com/afrah/QCPINN>.

Acknowledgments

We thank the National Center for High-Performance Computing of Türkiye for providing the computing resources and Bilimsel Araştırma Projeleri (BAP) Birimi, İstanbul Teknik Üniversitesi, for funding

this project under Grant Number MDK-2021-43019. We also thank the anonymous reviewers for their comments and feedback, which helped improve the manuscript.

Funding

A.F., and M.S.C. were funded by the Bilimsel Araştırma Projeleri (BAP) Birimi, İstanbul Teknik Üniversitesi, under Grant Number MDK-2021-43019.

Appendix A. Loss function design for PDEs

A.1. Helmholtz equation

The Helmholtz equation represents a fundamental PDE in mathematical physics, arising as the time-independent form of the wave equation. We consider a two-dimensional Helmholtz equation of the form:

$$\begin{aligned}\Delta u(x, y) + k^2 u(x, y) &= f(x, y) & (x, y) \in \Omega \\ u(x, y) &= h(x, y) & (x, y) \in \Gamma_0\end{aligned}\quad (5)$$

where, Δ is the Laplacian operator defined as $\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$, k is the wavenumber (related to the frequency of oscillation), $f(x, y)$ is the forcing term (source function), and specifies the Dirichlet boundary conditions. In this study, we choose an exact solution of the form:

$$u(x, y) = \sin(a_1 \pi x) \sin(a_2 \pi y)$$

on the square domain $\Omega = [-1, 1] \times [-1, 1]$ with Wavenumber $k = 1$, first mode number $a_1 = 1$, and the second mode number $a_2 = 4$. This choice of solution leads to a corresponding source term:

$$f(x, y) = u(x, y) \left[k^2 - (a_1 \pi)^2 - (a_2 \pi)^2 \right].$$

To solve this problem using QCPINN, we construct a loss function that incorporates both the physical governing equation and the boundary conditions:

$$\begin{aligned}\mathcal{L}(\theta) &= \min_{\theta} (\lambda_1 \|\mathcal{L}_{\text{phy}}(\theta)\|_{\Omega} + \lambda_2 \|\mathcal{L}_{\text{bc}}(\theta)\|_{\Gamma_0}) \\ &= \min_{\theta} \left(\lambda_1 \text{MSE} \left(\underbrace{\|u_{\theta_{xx}}(x, y) + u_{\theta_{yy}}(x, y) + \alpha u_{\theta}(x, y)\|_{\Omega}}_{\text{PDE residual}} \right) + \text{MSE} \left(\underbrace{\lambda_2 \|u(x, y) - u_{\theta}(x, y)\|_{\Gamma_0}}_{\text{Boundary condition}} \right) \right)\end{aligned}$$

where, $\alpha = (a_1 \pi)^2 + (a_2 \pi)^2$ combines the mode numbers into a single parameter. The chosen loss weights are $\lambda_1 = 1.0$ and $\lambda_2 = 10.0$.

A.2. Time-dependent 2D lid-driven cavity problem

The lid-driven cavity flow is a classical CFD problem and a fundamental test case for many numerical methods, where flow is governed by the unsteady, incompressible Navier–Stokes equations. The momentum equation, continuity equation, initial condition, no-slip boundary on walls, and moving lid condition are defined, respectively, as :

$$\begin{aligned}\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) &= -\nabla p + \mu \nabla^2 \mathbf{u} \\ \nabla \cdot \mathbf{u} &= 0 \\ \mathbf{u}(0, \mathbf{x}) &= 0 \\ \mathbf{u}(t, \mathbf{x}_0) &= 0 & \mathbf{x} \in \Gamma_0 \\ \mathbf{u}(t, \mathbf{x}_l) &= 1 & \mathbf{x} \in \Gamma_1\end{aligned}\quad (6)$$

where, the computational domain is $\Omega = (0, 1) \times (0, 1)$, the spatial discretization, $(N_x, N_y) = (100, 100)$ is uniform grid, with temporal domain $t \in [0, 10]$ seconds and $\Delta t = 0.01$ s with density $\rho = 1056 \text{ kg m}^{-3}$, viscosity $\mu = 1/\text{Re} = 0.01 \text{ kg (m}\cdot\text{s)}$, where Re is the Reynolds number. Γ_1 is the top boundary (moving lid) with tangential velocity $U = 1 \text{ m s}^{-1}$, and Γ_0 is the remaining three sides with no-slip condition ($\mathbf{u} = 0$) where $\mathbf{u} = (u, v)$.

For validation purposes, we compare the NN approximation with results obtained using the finite volume method. The PINN approach employs a composite loss function:

$$\mathcal{L}(\theta) = \min_{\theta} \left[\lambda_1 \underbrace{\|\mathcal{L}_{\text{phy}}(\theta)\|_{\Omega}}_{\text{PDE residual}} + \lambda_2 \underbrace{\|\mathcal{L}_{\text{up}}(\theta) + \mathcal{L}_{\text{bc1}}(\theta)\|_{\Gamma_1 \cup \Gamma_0}}_{\text{Boundary conditions}} + \lambda_3 \underbrace{\|\mathcal{L}_{\text{u0}}(\theta)\|_{\Omega}}_{\text{Initial conditions}} \right]$$

where, the physics-informed loss component, $\mathcal{L}_{\text{phy}}(\theta)$ consists of three terms:

$$\mathcal{L}_{\text{phy}}(\theta) = \mathcal{L}_{r_u} + \mathcal{L}_{r_v} + \mathcal{L}_{r_c}$$

such that:

$$\begin{aligned} \mathcal{L}_{r_u}(\theta) &= \text{MSE} \left[\left(u_{\theta_t} + u_{\theta} u_{\theta_x} + v_{\theta} u_{\theta_y} \right) + \frac{1.0}{\rho} p_{\theta_x} - \mu (u_{\theta_{xx}} + u_{\theta_{yy}}) \right] \\ \mathcal{L}_{r_v}(\theta) &= \text{MSE} \left[\left(v_{\theta_t} + u_{\theta} v_{\theta_x} + v_{\theta} v_{\theta_y} \right) + \frac{1.0}{\rho} p_{\theta_y} - \mu (v_{\theta_{xx}} + v_{\theta_{yy}}) \right] \\ \mathcal{L}_{r_c}(\theta) &= \text{MSE} [u_{\theta_x} + v_{\theta_y}] \end{aligned}$$

The boundary and initial conditions are enforced through the following:

$$\begin{aligned} \mathcal{L}_{\text{up}} &= \text{MSE} [(1.0 - \hat{u}) + \hat{v}] \\ \mathcal{L}_{\text{bc1}} = \mathcal{L}_{\text{bottom, right, left}} &= \text{MSE} [\hat{u} + \hat{v}] \\ \mathcal{L}_{\text{u0}} &= \text{MSE} [\hat{u} + \hat{v} + \hat{p}] \end{aligned}$$

where, $\mathcal{L}_{\text{up}}$ is the moving lid, $\mathcal{L}_{\text{bc1}}$ is the no-slip walls, and $\mathcal{L}_{\text{u0}}$ is the initial conditions. The loss weights are empirically chosen as $\lambda_1 = 0.1$, $\lambda_2 = 2.0$, $\lambda_2 = 2.0$, $\lambda_3 = 4.0$ for $\mathcal{L}_{\text{phy}}$, $\mathcal{L}_{\text{up}}$, $\mathcal{L}_{\text{bc1}}$ and $\mathcal{L}_{\text{u0}}$ respectively.

A.3. 1D Wave equation

The wave equation is a fundamental second-order hyperbolic PDE that models various physical phenomena. In its one-dimensional form, it describes the evolution of a disturbance along a single spatial dimension over time. The general form of the time-dependent 1D wave equation is:

$$\begin{aligned} u_{tt}(t, x) - c^2 u_{xx}(t, x) &= 0 \\ u(t, x_0) &= f_1(t, x) \quad (t, x) \text{ on } \Gamma_0 \\ u(t, x_1) &= f_2(t, x) \quad (t, x) \text{ on } \Gamma_1 \\ u(0, x) &= g(t, x) \quad (t, x) \in \Omega \\ u_t(0, x) &= h(t, x) \quad (t, x) \in \partial\Omega \end{aligned} \tag{7}$$

where, $u(t, x)$ represents the wave amplitude at position x and time t , and c is the wave speed characterizing the medium's properties. The subscripts denote partial derivatives: u_{tt} is the second time derivative, and u_{xx} is the second spatial derivative.

For our numerical investigation, we consider a specific case with the following parameters: $c = 2$, $a = 0.5$, $f_1 = f_2 = 0$. The exact solution is chosen as:

$$u(t, x) = \sin(\pi x) \cos(c\pi t) + 0.5 \sin(2c\pi x) \cos(4c\pi t)$$

This solution represents a superposition of two standing waves with different spatial and temporal frequencies. The problem is defined on the unit square domain $(t, x) \in [0, 1] \times [0, 1]$, leading to the specific boundary value problem:

$$\begin{aligned} u_{tt}(t, x) - 4u_{xx}(t, x) &= 0 \quad (t, x) \in \Omega = [0, 1] \times [0, 1] \\ u(t, 0) &= u(t, 1) = 0 \\ u(0, x) &= \sin(\pi x) + 0.5 \sin(4\pi x) \quad x \in [0, 1] \\ u_t(0, x) &= 0 \end{aligned}$$

To solve this problem using the proposed hybrid QCPINN, we construct a loss function that incorporates the physical constraints, boundary conditions, and initial conditions:

$$\begin{aligned}\mathcal{L}(\theta) &= \min_{\theta} [\lambda_1 \|\mathcal{L}_{\text{phy}}(\theta)\|_{\Omega} + \lambda_2 \|\mathcal{L}_{\text{bc}}(\theta)\|_{\Gamma_1} + \lambda_3 \|\mathcal{L}_{\text{ic}}(\theta)\|_{\Gamma_0}] \\ &= \min_{\theta} \left[\lambda_1 \text{MSE} \left(\underbrace{\|u_{\theta_{tt}}(t, x) - 4u_{\theta_{xx}}(t, x)\|_{\Omega}}_{\text{PDE residual}} \right) \right. \\ &\quad + \lambda_2 \text{MSE} \left(\underbrace{\|u_{\theta}(t, 0) + u_{\theta}(t, 1) + u_{\theta}(0, x) - \sin(\pi x) - 0.5 \sin(4\pi x)\|_{\Gamma_0 \cup \Gamma_1}}_{\text{Boundary/initial conditions}} \right) \\ &\quad \left. + \lambda_3 \text{MSE} \left(\underbrace{\|u_{\theta_t}(0, x)\|_{\partial\Omega}}_{\text{Initial velocity}} \right) \right]\end{aligned}$$

The loss weights are empirically chosen as $\lambda_1 = 0.1$, $\lambda_2 = 10.0$ and $\lambda_3 = 0.1$.

A.4. Klein–Gordon equation

The Klein–Gordon equation is a significant second-order hyperbolic PDE that emerges in numerous theoretical physics and applied mathematics areas. The equation represents a natural relativistic extension of the Schrödinger equation. In this study, we consider the one-dimensional nonlinear Klein–Gordon equation of the form:

$$\begin{aligned}u_{tt} - \alpha u_{xx} + \beta u + \gamma u^k &= f(t, x) & (t, x) \in \Omega \\ u(t, x) &= g_1(t, x) & (t, x) \text{ on } \Gamma_0 \\ u_t(t, x) &= g_2(t, x) & (t, x) \text{ on } \Gamma_1 \\ u(0, x) &= h(t, x) & (t, x) \in \partial\Omega \times [0, T]\end{aligned}\quad (8)$$

where, α is the wave speed coefficient, β is the linear term coefficient, γ is the nonlinear term coefficient, and k is the nonlinearity power. For our numerical study, we set $\alpha = 1$, $\beta = 0$, $\gamma = 1$, and $k = 3$. We choose an exact solution of the form:

$$u(t, x) = x \cos(5\pi t) + (tx)^3$$

This solution combines oscillatory behavior with polynomial growth. The complete boundary value problem on the unit square domain becomes:

$$\begin{aligned}u_{tt}(t, x) - u_{xx}(t, x) + u^3(t, x) &= x(-25\pi^2 \cos(5\pi t) - 6t^3) \\ &\quad + 6tx^3 + x^3(\cos(5\pi t) + t^3x^2)^3 \quad (t, x) \in \Omega = [0, 1] \times [0, 1] \\ u(t, 0) &= 0 \\ u(t, 1) &= \cos(5\pi t) + t^3 \\ u(0, x) &= x \\ u_t(0, x) &= 0\end{aligned}$$

To solve this nonlinear PDE using the proposed QCPINN, we construct a composite loss function incorporating the physical constraints, boundary conditions, and initial conditions:

$$\begin{aligned}\mathcal{L}(\theta) &= \min_{\theta} [\lambda_1 \|\mathcal{L}_{\text{phy}}(\theta)\|_{\Omega} + \lambda_2 \|\mathcal{L}_{\text{bc}}(\theta)\|_{\Gamma_1} + \lambda_3 \|\mathcal{L}_{\text{ic}}(\theta)\|_{\Gamma_0}] \\ &= \min_{\theta} \left[\lambda_1 \text{MSE} \left(\underbrace{\|u_{\theta_{tt}}(t, x) - u_{\theta_{xx}}(t, x) + u^3(t, x) - f(t, x)\|_{\Omega}}_{\text{PDE residual}} \right) \right. \\ &\quad + \lambda_2 \text{MSE} \left(\underbrace{\|u(t, 0) + u(t, 1) - \cos(5\pi t) + t^3 + u(0, x) - x\|_{\Gamma_1}}_{\text{Boundary/initial conditions}} \right) \\ &\quad \left. + \lambda_3 \text{MSE} \left(\underbrace{\|u_{\theta_t}(0, x)\|_{\partial\Omega}}_{\text{Initial velocity}} \right) \right]\end{aligned}$$

where, $f(t, x) = x(-25\pi^2 \cos(5\pi t) - 6t^3) + 6tx^3 + x^3(\cos(5\pi t) + t^3x^2)^3$. The loss weights are chosen empirically to balance the different components as $\lambda_1 = 1.0$, $\lambda_2 = 10.0$, and $\lambda_3 = 1.0$.

A.5. Convection–diffusion equation

The problem focuses on solving the 2D convection–diffusion equation, a PDE that models the transport of a quantity under convection (bulk movement) and diffusion (spreading due to gradients). The specific form considered here includes a viscous equation of the form:

$$\begin{aligned} u_t + c_1 u_x + c_2 u_y - D \Delta u(t, \mathbf{x}) &= f(t, \mathbf{x}) & (t, \mathbf{x}) \in \Omega \\ u(t, \mathbf{x}_0) &= g_0(t, \mathbf{x}_0) & (t, \mathbf{x}_0) \in \Gamma_0 \\ u(t, \mathbf{x}_1) &= g_1(t, \mathbf{x}_1) & (t, \mathbf{x}_1) \in \Gamma_1 \\ u(0, \mathbf{x}) &= h(0, \mathbf{x}) & \mathbf{x} \in \partial\Omega \times [0, T] \end{aligned} \quad (9)$$

where, c_1 is the convection velocity in the x direction, c_2 is the convection velocity in the y direction, D is the diffusion coefficient, and $\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ is the Laplacian operator. For our numerical investigation, we choose an exact solution describing a Gaussian pulse:

$$u(t, x, y) = \exp\left(-100\left((x-0.5)^2 + (y-0.5)^2\right)\right) \exp(-t).$$

The corresponding initial condition is:

$$h(0, x, y) = \exp\left(-100\left((x-0.5)^2 + (y-0.5)^2\right)\right).$$

We choose $c_1 = c_2 = 1.0$, and $D = 0.01$. The complete initial-boundary value problem on the unit cube domain becomes:

$$\begin{aligned} u_t(t, x, y) + u_x(t, x, y) + u_y(t, x, y) - 0.01(u_{xx}(t, x, y) + u_{yy}(t, x, y)) &= f(t, x, y) \\ u(t, 0, y) &= \exp\left(-25 - 100(y-0.5)^2 - t\right), \\ u(t, 1, y) &= \exp\left(-25 - 100(y-0.5)^2 - t\right), \\ u(0, x, y) &= h(x, y) \end{aligned}$$

where, $(t, x, y) \in \Omega = [0, 1] \times [0, 1] \times [0, 1]$, $f(t, x, y) = \exp(-100((x-0.5)^2 + (y-0.5)^2) - t)[3 - 200(x-0.5) - 200(y-0.5) - 400((x-0.5)^2 + (y-0.5)^2)]$. To solve this problem using the proposed QCPINN model, we formulate a loss function that incorporates the physical governing equation, boundary conditions, and initial conditions:

$$\begin{aligned} \mathcal{L}(\theta) &= \min_{\theta} (\lambda_1 \|\mathcal{L}_{\text{phy}}(\theta)\|_{\Omega} + \lambda_2 \|\mathcal{L}_{\text{bc}}(\theta)\|_{\Gamma_1} + \lambda_3 \|\mathcal{L}_{\text{ic}}(\theta)\|_{\Gamma_0}) \\ &= \min_{\theta} \left[\lambda_1 \min \left(\underbrace{\|u_{\theta_t} + u_{\theta_x} + u_{\theta_y} - 0.01(u_{\theta_{xx}} + u_{\theta_{yy}}) - f\|_{\Omega}}_{\text{PDE residual}} \right) \right. \\ &\quad + \lambda_2 \min \left(\underbrace{\|u(t, 0, y) - \exp(-25 - 100(y-0.5)^2 - t)\|_{\Gamma_0}}_{\text{Boundary conditions}} \right. \\ &\quad \left. \left. + \|u(t, 1, y) - \exp(-25 - 100(y-0.5)^2 - t)\|_{\Gamma_1} \right) \right. \\ &\quad \left. + \lambda_3 \min \left(\underbrace{\|u_{\theta}(0, x, y) - h(x, y)\|_{\Omega_0}}_{\text{Initial conditions}} \right) \right] \end{aligned}$$

The loss weights are chosen to balance the different components $\lambda_1 = 1.0$, $\lambda_2 = 10.0$, and $\lambda_3 = 10.0$.

Appendix B. CV-Circuit QCPINN

B.1. Algorithm

Algorithm 1. CV-Circuit QCPINN algorithm supports three nonlinear operations: Kerr, cubic phase, and cross-Kerr with flexible measurement schemes: quadrature or number basis. See section 5.1 for training details and the source code [21] implementation (using PennyLane framework with Strawberry Fields Fock backend).

Require: Qumodes: num_qumodes, Layers: num_layers, Device: d , Cutoff: c , Input: Tensor x

Output: Tensor y

1: **Initialize Parameters:**

2: $\psi_1, \psi_2, \phi_1, \phi_2, \phi_s, \phi_d \sim \mathcal{N}(0, 0.01\pi)$ (interferometer), $\alpha, r \sim \mathcal{N}(0, 0.001)$, $\Phi \sim \mathcal{N}(0, 0.001)$ (nonlinearity)

3: **Quantum Device:** Initialize d with num_qumodes, cutoff c .

4: **procedure** QuantumCircuitinput

5: **for** $i = 1$ to num_qumodes **do**

6: Encode Inputs with Displacements D : $D(\text{input}[i], 0)$

▷only real amplitudes

7: **end for**

8: **for** $l = 1$ to num_layers **do**

9: Interferometer($\psi_1[l], \phi_1[l]$)

10: Squeezing($r[l], \phi_s[l]$)

▷Phase-free: $\phi_s[l] = 0.0$

11: Interferometer($\psi_2[l], \phi_2[l]$)

12: Displacement($\alpha[l], \phi_d[l]$)

▷Phase-free: $\phi_d[l] = 0.0$

13: Nonlinearity Φ

▷Options: Kerr, Cubic Phase, Cross-Kerr

14: **end for**

15: Measure state for each wire

▷Measurement options: $\langle \hat{q}_i \rangle$ (Quadrature) or $\langle \hat{n}_i \rangle$ (Number)

16: **return** Measurements

17: **end procedure**

18: **procedure** Interferometer $\psi, \phi_{bs}[l]$

19: **for** $l = 1$ to num_qumodes **do**

20: **for** (q_1, q_2) in Pairs(num_qumodes) **do**

21: **if** $(l+k)\%2 \neq 1$ **then**

22: Beamsplitter $B(\psi[n], \phi_{bs}[n])$

▷Phase-free: $\phi_{bs}[n] = 0.0$

23: **end if**

24: **end for**

25: **end for**

26: **for** $i = 1$ to num_qumodes $- 1$ **do**

27: Rotation $R_Z(\psi[i])$

28: **end for**

29: **end procedure**

30: **procedure** ForwardPassInput x

31: $y \leftarrow$ QuantumCircuit(x)

32: **return** y

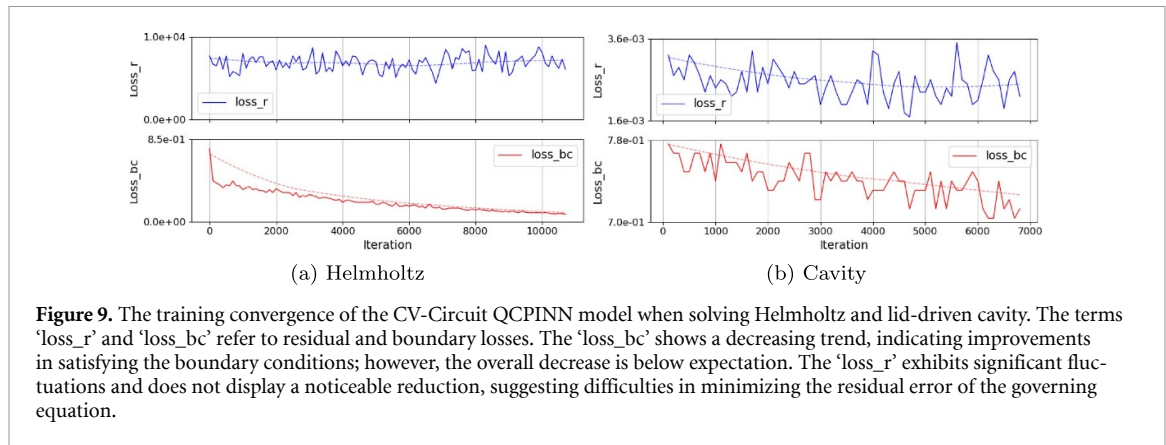
33: **end procedure**

B.2. Results

Table 4. The performance of CV-Circuit QCPINN models in solving the Helmholtz and lid-driven cavity equations. There are 12 combinations reported in table 1, however, results with the quadrature measurement scheme, Kerr nonlinear function, and phase-free parameterization are reported here. The remaining 11 combinations are not mentioned as the results are more or less similar. N_p : number of parameters, L_2 : relative L_2 error in %, $\mathcal{L}_{\text{final}}$: final loss. Training convergence is shown in figure 9.

PDE	N_p		L_2	$\mathcal{L}_{\text{final}}$
Helmholtz	469	u	$1.00e + 02$	5077.79
		f	$1.00e + 02$	
Cavity	621	u	$9.345e + 01$	0.102
		v	$9.979e + 01$	
		p	$8.620e + 01$	

Helmholtz equation. For the Helmholtz equation (5), we used 469 trainable parameters. The results, in table 4, show a high relative L_2 error for both the u and f variables, along with a significant final loss.



The training convergence pattern in figure 9(a) highlights some challenges. For example, the boundary loss, ‘loss_bc’, shows a steady decrease over 10 000 iterations, suggesting that the model is gradually improving in meeting the boundary conditions. However, the residual loss, ‘loss_r’, remains consistently high, with substantial fluctuations throughout the training process. This indicates that the model was struggling to satisfy the differential equation itself.

Time-dependent 2D lid-driven cavity equation. For the more complex lid-driven cavity equation (6), the CV-Circuit QCPINN model was implemented with 621 trainable parameters. The result, in table 4, shows that the relative L_2 errors for both velocity and pressure variables remain high. However, the final loss value of 0.102 is significantly lower than that observed for the Helmholtz equation. Figure 9(b) shows the training convergence, where the boundary loss exhibits some improvement, although it still experiences considerable fluctuations. The residual loss displays even greater oscillations compared to the Helmholtz case, indicating a greater challenge in accurately capturing the complex fluid dynamics governed by the Navier–Stokes equations.

This behavior is primarily attributed to gradient decay issues intrinsic to our CV-Circuit implementation. The vanishing gradient problem [88] manifests more severely due to the additional constraints imposed by quantum operations. Small initialization values, while necessary to prevent divergence in quantum state evolution, significantly diminish gradients during backpropagation.

We examined various techniques, such as Xavier initialization for learnable parameters, gradient clipping, and skip connections, to mitigate the problem of gradient decay, but these attempts proved ineffective. Exploring alternative optimization strategies beyond standard gradient descent may be beneficial in addressing this issue, though this remains beyond the scope of this work.

Appendix C. DV-Circuit QCPINN

Here, we present the results of additional DV-Circuit QCPINN models.

Table 5. Results from all configurations of QCPINN (DV-Circuit) models and PINN models for solving the Helmholtz equation. N_p : number of trainable parameters, L_2 : relative error (in %), L_{final} : final loss, T : time per iteration (in sec), M : peak memory consumption (in MB), u : reference solution, and f source term.

	Embedding	Topology	N_p	L_2	L_{final}	T	M
QCPINN (DV-Circuit)	Amplitude	Alternate	772	u 31.98 $\pm$ 9.51	24.42 $\pm$ 10.91	0.26 $\pm$ 0.00	420.0 $\pm$ 0.0
				f 7.70 $\pm$ 1.42			
		Cascade	771	u 21.33 $\pm$ 8.19	35.49 $\pm$ 29.26	0.28 $\pm$ 0.00	420.0 $\pm$ 0.0
				f 8.03 $\pm$ 2.83			
		Cross-mesh	796	u 29.39 $\pm$ 17.84	23.29 $\pm$ 16.08	0.31 $\pm$ 0.0	440.0 $\pm$ 0.0
				f 7.30 $\pm$ 3.21			
		Layered	776	u 20.95 $\pm$ 7.46	18.16 $\pm$ 9.18	0.29 $\pm$ 0.03	420.0 $\pm$ 0.0
				f 4.96 $\pm$ 0.92			
	Angle	Alternate	772	u 10.74 $\pm$ 5.54	4.87 $\pm$ 3.19	0.18 $\pm$ 0.0	420.0 $\pm$ 0.0
				f 3.42 $\pm$ 0.8			
		Cascade	771	u 6.69 $\pm$ 1.51	4.52 $\pm$ 1.86	0.17 $\pm$ 0.00	420.0 $\pm$ 0.0
				f 2.86 $\pm$ 0.54			
		Cross-mesh	796	u 7.24 $\pm$ 1.45	3.69 $\pm$ 1.30	0.25 $\pm$ 0.02	430.0 $\pm$ 0.0
				f 3.09 $\pm$ 0.70			
		Layered	776	u 18.19 $\pm$ 10.24	11.63 $\pm$ 8.73	0.19 $\pm$ 0.00	420.0 $\pm$ 0.0
				f 4.88 $\pm$ 1.42			
PINN		Model-1	7851	u 21.24 $\pm$ 12.13	16.62 $\pm$ 13.85	0.0041 $\pm$ 0.0	340.0 $\pm$ 0.0
				f 5.59 $\pm$ 2.56			
		Model-2	2751	u 12.12 $\pm$ 7.17	5.04 $\pm$ 3.12	0.0027 $\pm$ 0.0	340.0 $\pm$ 0.0
				f 3.23 $\pm$ 1.19			

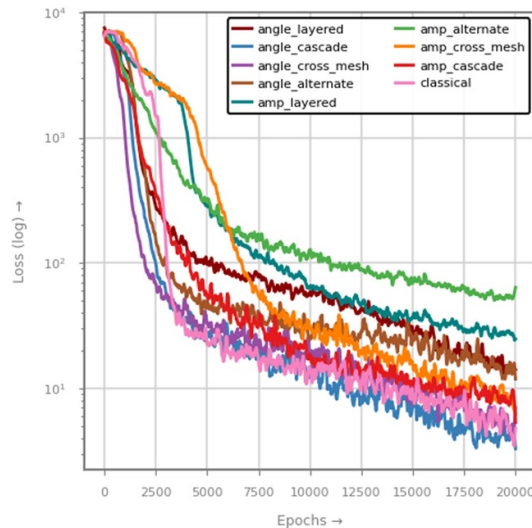


Figure 10. Training convergence history of all QCPINN (DV-Circuit) models compared to the best classical PINN model when solving the Helmholtz equation.

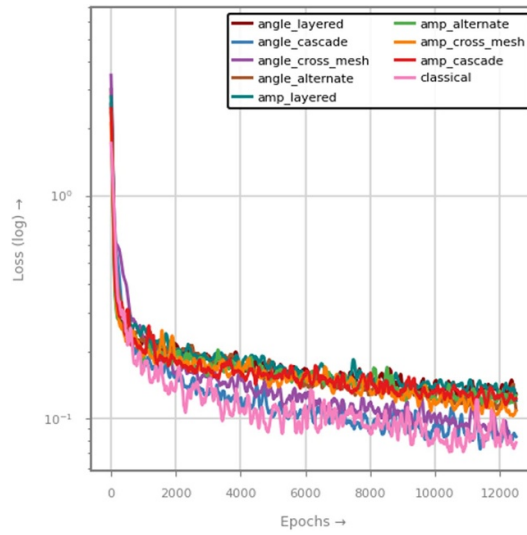
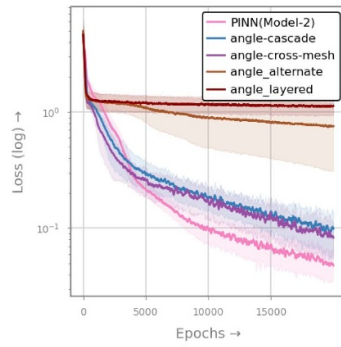
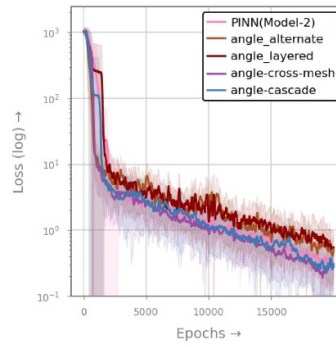


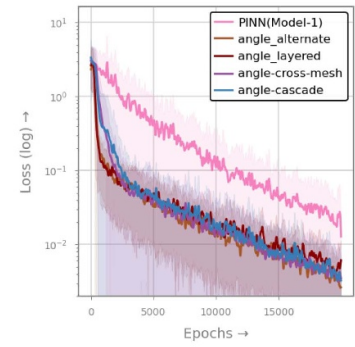
Figure 11. Training convergence history of all QCPINN (DV-Circuit) models compared to the best classical PINN model when solving the cavity problem.



(a) Wave



(b) Klein-Gordon



(c) Convection-diffusion

Figure 12. Each plot compares the convergence behavior of the QCPINN models with angle embedding with the best-performing classical PINN model.

Table 6. Results from all configurations of QCPINN (DV-Circuit) models and PINN models for solving the cavity problem. N_p : number of trainable parameters, L_2 : relative error (in %), L_{final} : final loss, T : time per iteration (in sec), M : peak memory consumption (in MB), u and v : velocity, and p : pressure.

	Embedding	Topology	N_p		L_2	$\mathcal{L}_{\text{final}}$	T	M
QCPINN	Amplitude	Alternate	924	u	54.40 ± 7.56	0.08 ± 0.01	0.57 ± 0.0	570.0 ± 0.0
				v	83.07 ± 1.04			
				p	55.53 ± 4.38			
		Cascade	928	u	59.70 ± 2.61	0.10 ± 0.03	0.69 ± 0.07	580.0 ± 0.0
				v	80.15 ± 3.38			
				p	56.44 ± 7.54			
		Cross-mesh	948	u	39.95 ± 0.00	0.07 ± 0.00	0.65 ± 0.0	620.0 ± 0.01
				v	49.62 ± 1.10			
				p	57.13 ± 2.00			
		Layered	928	u	59.92 ± 3.08	0.09 ± 0.03	0.60 ± 0.02	590.0 ± 0.0
				v	82.67 ± 3.33			
				p	51.92 ± 1.99			
	Angle	Alternate	924	u	26.13 ± 2.00	0.05 ± 0.00	0.37 ± 0.0	540.0 ± 0.0
				v	43.86 ± 9.00			
				p	43.86 ± 4.10			
		Cascade	923	u	25.73 ± 2.49	0.10 ± 0.02	0.36 ± 0.0	540.0 ± 0.0
				v	38.00 ± 3.76			
				p	42.65 ± 8.69			
		Cross-mesh	948	u	27.30 ± 4.87	0.08 ± 0.03	0.47 ± 0.0	590.0 ± 0.0
				v	39.88 ± 4.58			
				p	44.99 ± 9.24			
		Layered	928	u	59.87 ± 11.71	0.1 ± 0.17	4.01 ± 0.02	560.0 ± 0.0
				v	85.41 ± 8.22			
				p	73.36 ± 13.45			
PINN	Model-1		8003	u	21.61 ± 6.68	0.06 ± 0.02	0.01 ± 0.0	440.0 ± 0.0
				v	34.24 ± 2.76			
				p	34.82 ± 5.25			
	Model-2		2903	u	23.03 ± 2.45	0.06 ± 0.02	0.01 ± 0.0	435 ± 0.0
				v	38.81 ± 2.73			
				p	71.51 ± 4.79			

Table 7. Results from selected configurations of QCPINN (DV-Circuit) models and two PINN models. N_p : number of trainable parameters, L_2 : relative error (in %), L_{final} : final loss, T : time per iteration (in sec), M : peak memory consumption (in MB), u : velocity, and f : source terms.

PDEs	Method		N_p		L_2	$\mathcal{L}_{\text{final}}$	T	M
Wave equation (7)	QCPINN	Angle-Alternate	772	u	39.96 ± 7.85	0.78 ± 0.46	0.15 ± 0.0	405.0 ± 0.0
		Angle-Layered	776	u	44.76 ± 0.28	1.13 ± 0.26	0.17 ± 0.0	419.0 ± 0.0
		Angle-Cascade	771	u	16.74 ± 5.41	0.10 ± 0.03	0.25 ± 0.06	446.0 ± 0.0
	PINN	Angle-Cross-mesh	796	u	12.44 ± 4.42	0.09 ± 0.03	0.38 ± 0.08	469.0 ± 0.0
		Model-1	7851	u	20.94 ± 1.70	0.15 ± 0.04	0.02 ± 0.03	353.0 ± 0.0
		Model-2	2751	u	11.13 ± 1.78	0.05 ± 0.01	0.02 ± 0.02	351 ± 0.00
Klein–Gordon equation (8)	QCPINN	Angle-Alternate	772	u	15.54 ± 5.56	0.51 ± 0.37	0.16 ± 0.0	406.0 ± 0.0
				f	1.99 ± 0.96			
		Angle-Layered	776	u	14.86 ± 4.78	0.51 ± 0.41	0.17 ± 0.0	409.0 ± 0.0
				f	2.33 ± 1.10			
		Angle-Cascade	771	u	4.24 ± 1.88	0.27 ± 0.10	0.25 ± 0.03	447.0 ± 0.0
				f	1.72 ± 0.50			
	PINN	Angle-Cross-mesh	796	u	5.64 ± 1.87	0.34 ± 0.21	0.42 ± 0.11	468.0 ± 0.0
				f	1.65 ± 0.38			
		Model-1	7851	u	5.47 ± 3.57	0.61 ± 0.35	0.03 ± 0.04	348.0 ± 0.0
				f	2.58 ± 1.24			
Convection–diffusion equation (9)	QCPINN	Model-2	2751	u	4.48 ± 1.72	0.34 ± 0.13	0.02 ± 0.02	350.0 ± 0.0
				f	1.79 ± 0.47			
		Angle-Alternate	822	u	5.06 ± 2.08	0.001 ± 0.001	0.13 ± 0.0	403.0 ± 0.0
				f	3.22 ± 0.65			
		Angle-Layered	826	u	5.10 ± 3.22	0.01 ± 0.01	0.15 ± 0.0	410.0 ± 0.0
				f	3.40 ± 1.62			
	PINN	Angle-Cascade	821	u	5.04 ± 1.43	0.001 ± 0.001	0.27 ± 0.06	445.0 ± 0.0
				f	3.03 ± 1.00			
		Angle-Cross-mesh	846	u	4.54 ± 2.17	0.001 ± 0.001	0.42 ± 0.11	468.0 ± 0.0
				f	2.63 ± 1.61			
	PINN	Model-1	7901	u	12.62 ± 6.05	0.02 ± 0.02	0.01 ± 0.0	348.0 ± 0.0
				f	5.57 ± 3.05			
	PINN	Model-2	2801	u	36.80 ± 8.79	0.11 ± 0.06	0.01 ± 0.0	346.0 ± 0.0
				f	21.27 ± 8.12			

ORCID iDs

Afrah Farea  0000-0003-4412-5377

Saiful Khan  0000-0002-6796-5670

Mustafa Serdar Celebi  0000-0003-4566-0216

References

- [1] Karniadakis G E, Kevrekidis I G, Lu L, Perdikaris P, Wang S and Yang L 2021 Physics-informed machine learning *Nat. Rev. Phys.* **3** 422–40
- [2] Cai S, Mao Z, Wang Z, Yin M and Karniadakis G E 2021 Physics-informed neural networks (PINNs) for fluid mechanics: a review *Acta Mech. Sin.* **37** 1727–38
- [3] Farea A, Khan S, Daryani R, Ersan E C, and Celebi M S 2025 Learning fluid-structure interaction dynamics with physics-informed neural networks and immersed boundary methods (arXiv:2505.18565)
- [4] Farea A, Khan S and Celebi M S 2025 Multi-objective loss balancing in physics-informed neural networks for fluid flow applications *32nd IEEE Int. Conf. on High Performance Computing, Data and Analytics* (Accepted)
- [5] Farea A and Çelebi M S 2023 On the evaluation of general sparse hybrid linear solvers *Numer. Linear Algebra Appl.* **30** e2469
- [6] Hu Z, Shukla K, Karniadakis G E and Kawaguchi K 2024 Tackling the curse of dimensionality with physics-informed neural networks *Neural Netw.* **176** 106369
- [7] Hu Z, Yang Z, Wang Y, Karniadakis G E and Kawaguchi K 2025 Bias-variance trade-off in physics-informed neural networks with randomized smoothing for high-dimensional PDEs *SIAM J. Sci. Comput.* **47** C846–72
- [8] Ying X 2019 An overview of overfitting and its solutions *J. Physics: Conf. Ser.* **1168** 022022
- [9] Farea A and Celebi M S 2025 Learnable activation functions in physics-informed neural networks for solving partial differential equations *Comput. Phys. Commun.* **315** 109753
- [10] Mitarai K, Negoro M, Kitagawa M and Fujii K 2018 Quantum circuit learning *Phys. Rev. A* **98** 032309
- [11] Havlíček V, Córcoles A D, Temme K, Harrow A W, Kandala A, Chow J M and Gambetta J M 2019 Supervised learning with quantum-enhanced feature spaces *Nature* **567** 209–12
- [12] Cerezo M et al 2021 Variational quantum algorithms *Nat. Rev. Phys.* **3** 625–44
- [13] Knudsen M and Mendl C B 2020 Solving Differential equations via continuous-variable quantum computers (arXiv:2206.14184)

- [14] Schuld M, Sweke R and Meyer J J 2021 Effect of data encoding on the expressive power of variational quantum-machine-learning models *Phys. Rev. A* **103** 032430
- [15] Abbas A, Sutter D, Zoufal C, Lucchi A, Figalli A and Woerner S 2021 The power of quantum neural networks *Nat. Comput. Sci.* **1** 403–9
- [16] Du Y, Hsieh M-H, Liu T, You S and Tao D 2021 Learnability of quantum neural networks *PRX Quantum* **2** 040337
- [17] Sedykh A, Podapaka M, Sagingalieva A, Pinto K, Pflitsch M and Melnikov A 2024 Hybrid quantum physics-informed neural networks for simulating computational fluid dynamics in complex shapes *Mach. Learn.: Sci. Technol.* **5** 025045
- [18] Dehaghani N B, Pedro Aguiar A and Wisniewski R 2024 A hybrid quantum-classical physics-informed neural network architecture for solving quantum optimal control problems 2024 *IEEE Int. Conf. on Quantum Computing and Engineering (QCE)* vol 1 (IEEE) pp 1378–86
- [19] Trahan C, Loveland M and Dent S 2024 Quantum physics-informed neural networks *Entropy* **26** 649
- [20] Leong F Y, Ewe W-B, Tran S B Q, Zhang Z and Khoo J Y 2025 Hybrid quantum physics-informed neural network: towards efficient learning of high-speed flows *Comput. Fluids* **301** 106782
- [21] Farea A 2025 QCPINN: quantum classical physics-informed neural networks for solving PDEs(<https://github.com/afrah/QCPINN>) (Accessed 13 October 2025)
- [22] Harrow A W, Hassidim A and Lloyd S 2009 Quantum algorithm for linear systems of equations *Phys. Rev. Lett.* **103** 150502
- [23] Ambainis A 2012 Variable time amplitude amplification and quantum algorithms for linear algebra problems *Symp. on Theoretical Aspects of Computer Science (Paris, France)* vol 14, ed T W C Dürre pp 636–47
- [24] Somma R, Childs A and Kothari R 2016 Quantum linear systems algorithm with exponentially improved dependence on precision *APS March Meeting Abstracts* vol 2016 (APS) pp H44–001
- [25] Leyton S K and Osborne T J 2008 A quantum algorithm to solve nonlinear differential equations (arXiv:0812.4423)
- [26] Berry D W 2014 High-order quantum algorithm for solving linear differential equations *J. Phys. A: Math. Theor.* **47** 105301
- [27] Childs A M and Liu J-P 2020 Quantum spectral methods for differential equations *Commun. Math. Phys.* **375** 1427–57
- [28] Berry D W, Childs A M, Ostrander A and Wang G 2017 Quantum algorithm for linear differential equations with exponentially improved dependence on precision *Commun. Math. Phys.* **356** 1057–81
- [29] Yepez J 2002 Quantum lattice-gas model for the Burgers equation *J. Stat. Phys.* **107** 203–24
- [30] Montanaro A and Pallister S 2016 Quantum algorithms and the finite element method *Phys. Rev. A* **93** 032324
- [31] Oz F, Vuppala R K S S, Kara K and Gaitan F 2022 Solving Burgers' equation with quantum computing *Quantum Inf. Process.* **21** 30
- [32] Oz F, San O and Kara K 2023 An efficient quantum partial differential equation solver with chebyshev points *Sci. Rep.* **13** 7767
- [33] Preskill J 1998 Reliable quantum computers *Proc. R. Soc. A* **454** 385–410
- [34] Terhal B M 2015 Quantum error correction for quantum memories *Rev. Mod. Phys.* **87** 307–46
- [35] Childs A M, Su Y, Tran M C, Wiebe N and Zhu S 2021 Theory of trotter error with commutator scaling *Phys. Rev. X* **11** 011020
- [36] Bravyi S and Kitaev A 2005 universal quantum computation with ideal Clifford gates and noisy ancillas *Phys. Rev. A* **71** 022316
- [37] Boixo S, Isakov S V, Smelyanskiy V N, Babbush R, Ding N, Jiang Z, Bremner M J, Martinis J M and Neven H 2018 Characterizing quantum supremacy in near-term devices *Nat. Phys.* **14** 595–600
- [38] Aaronson S 2015 Read the fine print *Nat. Phys.* **11** 291–3
- [39] Ciliberto C, Herbster M, Jalongo A D, Pontil M, Rocchetto A, Severini S and Wossnig L 2018 Quantum machine learning: a classical perspective *Proc. R. Soc. A* **474** 20170551
- [40] Lloyd S, Mohseni M and Rebentrost P 2013 Quantum algorithms for supervised and unsupervised machine learning (arXiv:1307.0411)
- [41] Pérez-Salinas A, López-Núñez D, García-Sáez A, Forn-Díaz P and Latorre J I 2021 One qubit as a universal approximant *Phys. Rev. A* **104** 012405
- [42] Goto T, Tran Q H and Nakajima K 2021 universal approximation property of quantum machine learning models in quantum-enhanced feature spaces *Phys. Rev. Lett.* **127** 090506
- [43] Liu Y, Arunachalam S and Temme K 2021 A rigorous and robust quantum speed-up in supervised machine learning *Nat. Phys.* **17** 1013–7
- [44] Killoran N, Bromley T R, Arrazola J M, Schuld M, Quesada N and Lloyd S 2019 Continuous-variable quantum neural networks *Phys. Rev. Res.* **1** 033063
- [45] Cao Y et al 2019 Quantum chemistry in the age of quantum computing *Chem. Rev.* **119** 10856–915
- [46] Bauer B, Bravyi S, Motta M and Chan G K-L 2020 Quantum algorithms for quantum chemistry and quantum materials science *Chem. Rev.* **120** 12685–717
- [47] Nachman B, Provasoli D, De Jong W A and Bauer C W 2021 Quantum algorithm for high energy physics simulations *Phys. Rev. Lett.* **126** 062001
- [48] Carlo A 2002 Trugenberger. quantum pattern recognition *Quantum Inf. Process.* **1** 471–93
- [49] Peruzzo A, McClean J, Shadbolt P, Yung M-H, Zhou X-Q, Love P J, Aspuru-Guzik A and O'Brien J L 2014 A variational eigenvalue solver on a photonic quantum processor *Nat. Commun.* **5** 4213
- [50] McClean J R, Romero J, Babbush R and Aspuru-Guzik A 2016 The theory of variational hybrid quantum-classical algorithms *New J. Phys.* **18** 023023
- [51] Bravo-Prieto C, LaRose R, Cerezo M, Subasi Y, Cincio L and Coles P J 2023 Variational quantum linear solver *Quantum* **7** 1188
- [52] Albino A S, Jardim L C, Knupp D C, Neto A J S, Pires O M and Nascimento E G S 2022 Solving partial differential equations on near-term quantum computers (arXiv:2208.05805)
- [53] Kyriienko O, Paine A E and Elfving V E 2021 Solving nonlinear differential equations with differentiable quantum circuits *Phys. Rev. A* **103** 052416
- [54] Bultrini D and Vendrell O 2023 Mixed quantum-classical dynamics for near-term quantum computers *Commun. Phys.* **6** 328
- [55] Paine A E, Elfving V E and Kyriienko O 2023 Quantum kernel methods for solving regression problems and differential equations *Phys. Rev. A* **107** 032428
- [56] Lubasch M, Joo J, Moinier P, Kiffner M and Jaksch D 2020 Variational quantum algorithms for nonlinear problems *Phys. Rev. A* **101** 010301
- [57] Braunstein S L and Van Loock P 2005 Quantum information with continuous variables *Rev. Mod. Phys.* **77** 513–77
- [58] Setty A, Abdusalamov R and Motzoi F 2025 Self-adaptive physics-informed quantum machine learning for solving differential equations *Mach. Learn.: Sci. Technol.* **6** 015002
- [59] Preskill J 2018 Quantum computing in the NISQ era and beyond *Quantum* **2** 79
- [60] Bharti K et al 2022 Noisy intermediate-scale quantum algorithms *Rev. Mod. Phys.* **94** 015004

- [61] Krishnapriyan A, Gholami A, Zhe S, Kirby R and Mahoney M W 2021 Characterizing possible failure modes in physics-informed neural networks *Advances in Neural Information Processing Systems* vol 34 pp 26548–60
- [62] Rishi J, Gafoor A, Kumar S and Subramani D 2024 On the Training Efficiency of Shallow Architectures for Physics Informed Neural Networks *Computational Science – ICCS 2024 (Lecture Notes in Computer Science)* vol 14834 (Springer) pp 363–77
- [63] Shin Y, Zhang Z and Karniadakis G E 2023 Error estimates of residual minimization using neural networks for linear PDEs *J. Mach. Lear. Model. Comput.* **4** 73–101
- [64] Bergholm V et al 2018 PennyLane: automatic differentiation of hybrid quantum-classical computations (arXiv:1811.04968)
- [65] Broughton M et al 2020 TensorFlow quantum: a software framework for quantum machine learning (arXiv:2003.02989)
- [66] Li Z, Kovachki N, Azizzadenesheli K, Liu B, Bhattacharya K, Stuart A and Anandkumar A 2021 Fourier neural operator for parametric partial differential equations *Int. Conf. on Learning Representations*
- [67] Raissi M, Perdikaris P and Karniadakis G E 2019 Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations *J. Comput. Phys.* **378** 686–707
- [68] Wang S, Teng Y and Perdikaris P 2021 Understanding and mitigating gradient flow pathologies in physics-informed neural networks *SIAM J. Sci. Comput.* **43** A3055–81
- [69] Vemuri S K and Denzler J 2023 Gradient statistics-based multi-objective optimization in physics-informed neural networks *Sensors* **23** 8665
- [70] Liu Q, Chu M and Thuerey N 2024 Config: towards Conflict-free Training of Physics Informed Neural Networks *The 13th Int. Conf. on Learning Representations*
- [71] Heydari A A, Thompson C A and Mehmood A 2019 Softadapt: Techniques for adaptive loss weighting of neural networks with multi-part loss functions (arXiv:1912.12355)
- [72] Bischof R and Kraus M A 2025 Multi-Objective loss balancing for physics-informed deep learning *Comput. Methods Appl. Mech. Eng.* **439** 117914
- [73] McClenny L D and Braga-Neto U M 2023 Self-adaptive physics-informed neural networks *J. Comput. Phys.* **474** 111722
- [74] Li S and Feng X 2022 Dynamic weight strategy of physics-informed neural networks for the 2D Navier–Stokes equations *Entropy* **24** 1254
- [75] Costabal F S, Yang Y, Perdikaris P, Hurtado D E and Kuhl E 2020 Physics-informed neural networks for cardiac activation mapping *Front. Phys.* **8** 42
- [76] Ren P, Rao C, Liu Y, Wang J-X and Sun H 2022 PhyCRNet: physics-informed convolutional-recurrent network for solving spatiotemporal PDEs *Comput. Methods Appl. Mech. Eng.* **389** 114399
- [77] Norambuena A, Mattheakis M, González F J and Coto R 2024 Physics-informed neural networks for quantum control *Phys. Rev. Lett.* **132** 010801
- [78] McClean J R, Boixo S, Smelyanskiy V N, Babbush R and Neven H 2018 Barren plateaus in quantum neural network training landscapes *Nat. Commun.* **9** 4812
- [79] Cerezo M, Sone A, Volkoff T, Cincio L and Coles P J 2021 Cost function dependent barren plateaus in shallow parametrized quantum circuits *Nat. Commun.* **12** 1791
- [80] Holmes Z, Sharma K, Cerezo M and Coles P J 2022 Connecting ansatz expressibility to gradient magnitudes and barren plateaus *PRX Quantum* **3** 010313
- [81] Tacchino F, Barkoutsos P, Macchiavello C, Tavernelli I, Gerace D and Bajoni D 2020 Quantum implementation of an artificial feed-forward neural network *Quantum Sci. Technol.* **5** 044010
- [82] Mangini S, Tacchino F, Gerace D, Bajoni D and Macchiavello C 2021 Quantum computing models for artificial neural networks *Europhys. Lett.* **134** 10002
- [83] Farhi E and Neven H 2018 Classification with quantum neural networks on near term processors (arXiv:1802.06002)
- [84] Sim S, Johnson P D and Aspuru-Guzik A 2019 Expressibility and entangling capability of parameterized quantum circuits for hybrid quantumclassical algorithms *Adv. Quantum Technol.* **2** 1900070
- [85] Jaderberg B, Agarwal A, Leonhardt K, Kiffner M and Jaksch D 2020 Minimum hardware requirements for hybrid quantum-classical DMFT *Quantum Sci. Technol.* **5** 034015
- [86] Mouton L, Reiter F, Chen Y and Rebentrost P 2024 Deep-learning-based quantum algorithms for solving nonlinear partial differential equations *Phys. Rev. A* **110** 022612
- [87] Killoran N, Izaac J, Quesada N, Bergholm V, Amy M and Weedbrook C 2019 Strawberry fields: a software platform for photonic quantum computing *Quantum* **3** 129
- [88] Glorot X and Bengio Y 2010 Understanding the difficulty of training deep feedforward neural networks *Proc. 13th Int. Conf. on Artificial Intelligence and Statistics (JMLR Workshop and Conf. Proc.)* pp 249–56
- [89] Lvovsky A I 2015 Squeezed light *Photonics: Sci. Found., Technol. Appl.* **1** 121–63
- [90] Wang X-B, Hiroshima T, Tomita A and Hayashi M 2007 Quantum information with Gaussian states *Phys. Rep.* **448** 1–111
- [91] Drummond P D and Walls D F 1980 Quantum theory of optical bistability: I. Nonlinear polarisability model *J. Phys. A: Math. Gen.* **13** 725
- [92] Li Z et al 2023 Error per single-qubit gate below 10^{-4} in a superconducting qubit *npj Quantum Inform.* **9** 111
- [93] Hyppä E et al 2024 Reducing leakage of single-qubit gates for superconducting quantum processors using analytical control pulse envelopes *PRX Quantum* **5** 030353
- [94] Ding L et al 2023 High-fidelity, frequency-flexible two-qubit fluxonium gates with a transmon coupler *Phys. Rev. X* **13** 031035
- [95] Li R, Kubo K, Ho Y, Yan Z, Nakamura Y and Goto H 2024 Realization of high-fidelity CZ gate based on a double-transmon coupler *Phys. Rev. X* **14** 041050